KomeCare design document. Hakeem Abdullahi

Note: this is my original design. Some decisions have changed during the build. The README has the current state of the project.

Contents

Analysis: 4

Context: 4

Background information: 4

The problem 5

- Problem 1 5
- Problem 2: 5
- Problem 3: 6
- Problem 4 6

SOLUTION: (USING COMPUTATIONAL THINKING) 6

- DECOMPOSITION 7
- ABSTRACTION 7
- PATTERN RECOGNITION: 7
- ALGORITHIMETIC THINKING: 8

STAKEHOLDERS: 8

- Primary user- clients 8
- SECONDARY USERS – Family Members 9
- The care provider (management) 9
- Care staff 10

DFD level 0- current system 12

- Explanation and justification of DFD 12

DFD level 0- proposed system 13

- Explanation and justification of DFD 13

Research into the problem 14

Primary research: I spoke informally with carers and older computer users about what they find difficult. This gave me a direct view of my target users and their needs. The downside is that it is time consuming and the number of people was small, so I have not presented it as statistics. .. 14

Secondary research: used published data from organization like action fraud and Age UK. This was quicker and showed that this problem extends beyond one care company and it is since as a national issue. However, the key thing I had to put in mind is that I have to be careful about using reputable sources since not all published research can be trusted or reliable or even up to date14

- What people told me: 14

- Secondary research**:** 14
- Existing solutions research : 16

CLIENT REQUIREMENTS: 21

- functional requirements: 21
- Requirements comparison table 23

Hardware requirements: 26

Software Requirements 27

Limitation of the proposed solution 27

SUCCESS CRITERIA 28

DESIGN 30

Breaking the problem down: 30

DFD level 0- proposed system 32

DATABASE AND ENTITY RELATIONSHIP DIAGRAMS: 32

Data Dictionaries and Validation: 35

USER: 35

- SETTINGS: 36
- VAULT: 38
- REMINDER: 40
- ACTIVITY LOG: 41

Usability Features: 43

Human computer interface: 44

Algorithms: 48

- Threat scanner flowchart: 48
- Password vault: 49
- 50
- Carer settings: 50

Algorithms: 51

- threat Scanner: 51
- Password vault: 52
- Carer settings Login 53

Testing: 54

- White box testing 54

Black Box Testing:..... 59
Functional Testing 61
End User testing 63

Analysis:

Context:

Elderly people aged 65 above mostly in their retiring staged are increasingly required to use computer for essential task such online banking, booking and receiving email appointments, and staying in touch with family. However, many find modern computer interface quite overwhelming maybe due to complex menu and design structure. At the same time, they are frequently targeted by scammer as they are more vulnerable due to lack of knowledge to modern day technology. The idea for this project came from a home-care company where a member of my family used to work. Carers there kept seeing their clients struggle with exactly these problems, so I decided to design an application that reduces these risks.

Background information:

The home-care company provided care to elderly clients and clients in need of extra support. Many of their clients lived independently and used a computer for essential tasks such as booking medical appointments, the NHS app, video calls or contacting friends and family.

However, the carer who visit the client at their various identified that the client has major challenges, such as:

- Modern day application software seem to have complicated design structure and arrangement of buttons and role for each button whether the icon of the button or the size , also the font size can be difficult for some user to read and understand maybe due to font style or complexity of the word or grammar, This means that It limit independence, increase isolation. Which create dependency on family members
- Also, elderly people are being targeted by modern scammer as they are more vulnerable. This means they receive suspicious email and call from them making them offer and situation they could easily fall into. This means that it could lead to some psychological impact often leading to loss of confidence, reluctance to use technology at all, and in some cases financial difficulties

Currently the carer has noticed due to this problem client are avoiding computer (limiting their independence), relying heavily on family members for basic task and help in which they may not be always available to help

The problem

From talking with people who work in home care, I learned about a significant problem affecting their clients' wellbeing and independence

- Problem 1: essential app interface complexity



elderly client report shows that window desktop and application are too cluttered and confusing for them due to:
Multiply and unique icon, and menus they don't understand. This led to frustration, Major mistake (clicking wrong thing) and ultimately avoiding computer because of the failed progress with them this is because the experience can be genuinely intimidating and confusing

For example : past work experience and recent contact with some carers describing what they observed, client often call their attention "I clicked something and now everything look different, is it broken?, or like some will say " there is too many things on the screens" this is because window are designed for general users across all age group and skill levels, a typical solution to this is book a class or a tech teacher who teach people who to use a computer or basic ICT usage, however this will cost more and more time which may lead to more frustration or unnecessary complexity

- Problem 2: **Vulnerability to scams**

Carers have documented several instances where their patient have fallen into victim of suspicious act online for example the most common one is he constant Pop-up "Your computer has virus" click to resolve or dial a number to call tech support to help resolve, resulting to compromised login details

Why is this problem more concerning?

This is because current tool available to protect users- antivirus software, browser security warning and spam filtering in email app, they all communicate in technical terms where some young adult might not understand and must reach out for further understanding yet alone elderly users.
Also some security software they may decide to use might be quite aggressive with some

urgent warning (such as constant multiple notification and the ringtone could f this notification could be quite overwhelming for elderly users.

- Problem 3: **Lack of integrated solution scheme**

  When researching this problem initially, I discovered that there is existing solution for some of these issues, such as some popular interface program like “ELDY” and security tool like various antivirus packages. The problem is implementing these solutions together is the challenging aspect. Because If a care provider wanted to provide both simplified interface and security protection. They will need to

  - Install a simplified interface program first
  - Plus install a security launcher program
  - Install a password manager
  - Plus configure all of these to work together
  - Plus, they may decide to train both client and carers how to use multiple different interfaces

  This completely ignore the purpose of simplification and add more complexity. The user must understand how to navigate between different interfaces, for someone who might be struggling with the basic computer use this can be quite overwhelming.

  This means that they need is an integrated solution, one program that provides both the simplicity and security, this is the gap that current solutions have failed to address

- Problem 4: **Reduced Independence**

  Due to these challenges, many clients have stopped using computer independently due to past failed experience with it, which means they require constant family and staff help which maybe not be available at the time when needed because they have become dependent on family member or staff for basic task such as checking bank balance, sending email, or looking up something online.

  Care staff saw this as a real issue affecting their clients' wellbeing, which is why I decided to develop a solution

  This problem also creates additional work for staff and family members, who find themselves having to provide tech support on top of their other responsibilities, this means that it is not a sustainable situation for anyone involved

SOLUTION: (USING COMPUTATIONAL THINKING)

I will create an integrated computer desktop application that addresses both problems simultaneously

Why is this problem solvable by computational methods?

This problem is highly amenable to a computational approach because it requires the automation of repetitive, all data will be stored digitally , complex task that manual (like written guides or IT classes) cannot solve efficiently. By overriding the normal window graphical user interface (GUI) and replacing it with a simplified launcher, a computational approach allows me to abstract the operating systems complexities away from the elderly use. Furthermore, identifying modern scams requires rapid data processing, algorithmic thinking and can be used to scan URLs and email test against databases of known threats in real time providing automated data validation that a human carer cannot constantly monitor. A software solution provides consistent, 24/7 support making it the most efficient way to increase user independence.

- DECOMPOSITION: was essential because the overall thinking problem "elderly struggled with computer and are vulnerable to scams" is way too large and vague to solve directly. So, by breaking it down into smaller, specific problems, I could identify exactly what needed to be addressed and why is it is needed to be address. For example,
  1. The window desktop is too complex - need a simplified launcher interface
  2. Users forget passwords - Need a simplified and secure password manager
  3. User don't know if website is safe - Need URL checking and protected browsing
  4. Users need a basic organization tool - Need a diary/ reminder
  - Breaking down the problem into lot of smaller problems like this is justified because it allows me to develop and test each component independently. I would constantly be using functions in order to break down my problem so it is easier to read and understand the code. Another of breaking this down separately is that if i run the program and an error occur or is detected if i get notified with the function where the error occur. This will save me time and i can work much more efficiently when coding. furthermore, leaving spaces between blocks of code to separate them from each other would make it easier for me read and understand what my code is doing
  - ABSTRACTION: The principle of abstraction is about removing unnecessary details and focusing on what is essential. When i looked at a standard window desktop.

  They don't need: access to system settings, command prompt, device manager, file explorer, or dozen of all other utility. What they actually need is much simpler, a way to browse the web safely, check message, manage passwords, and keep track of reminders.
  By reducing the option to only what is necessary, I'm working with human cognitive limitations rather than against them
  The justification for this approach comes directly from my research – when there are too many options, elderly users become overwhelmed and either make mistakes or avoid using the system altogether, abstraction solve this by ensuring the interface only present what's genuinely needed

- PATTERN RECOGNITION: helped me identify common method in scam attempts. After researching common scams that target elderly people, i notice patterns such as
  - Phishing emails almost always create false urgency ("your account will be closed in 24 hours!")

- They often impersonate trusted organizations (banks, NHS, HMRC, Royal mail)
- They contain suspicious links with misspelled domains (amazOn,com instead of amazon.com)

Recognizing these patterns is valuable because it means i can program system to check for this specific indicator. I just need to check for these common patterns; the system can flag the messages as suspicious with high confidence

- ALGORITHIMETIC THINKING: involves designing step- by - step processes to solve problems for my project, this means planning out exactly how the system will respond to different inputs and situations. Thinking algorithmically like this is justified because it forces me to consider every possible scenario and edge case Before i start coding. It's much easier to identify potential problems at planning stage than to discover most of them halfway through development and when fixing them it might require rewriting large sections of code
in summary, I will apply algorithmic thinking by converting my system requirements into logical flowcharts. This allowed be to define the exact relationship between input and output. This approach is vital to ensuring software robustness, as it allowed me to map out how the software handle unexpected users' behaviour before I committed to a specific programming language or database structure.

STAKEHOLDERS:

- Primary user- clients

Elderly people aged 60+ who receive home care. They either live independently in their own home or in a supported environment. The majority are aged 65 – 80, each varying in level of physical ability and cognitive function, though all have the mental capacity to use computers with appropriate support.

Their needs from my software:

- Require minimal prior knowledge – they cannot be expected to understand concept like operating systems, applications and file structure
- Provide clear visual feedback – large text and clear button label are essential. Many elderly users have declining eyesight, so accessibility must be built in from the start
- Protect them from threats without requiring security expertise- this is because of them many not know some social engineering technique such as phishing or how identify a fake website. The system should protect them automatically and only involve them when absolutely necessary, with explanations plain English
- Maintaining their respectability: the interface should not be childish. These are adult with decades of life experience who deserve a system that treat them with respect even while simplifying technical complexity

- **Their IT skills**

From initial conversation with care staff and my own observation, vary considerably, but generally sit in the low to moderate range. Most can perform very basic tasks like turning on a computer and using a mouse, but struggle with anything beyond that. Many of the rest are what that could be term as reluctant users" they understand that they need to use technology to access certain services, but they find it stressful and confusing

- **How they will use it:**
  Turn on computer software launches automatically, larger clear button for each task
  Click on the task they want to do
  Complete their task within that tool
  Task – built- in protection works invisibly in the background
  Return to the main menu if they want to or
  Shut down computer via software button

- SECONDARY USERS – Family Members

Who they are:

Adult children, grandchildren and relative of the elderly clients.

- Their needs:
  Easy initial setup and configuration
  Confidence that their relatives are protected and less worries and constant checking giving them extra time to do their other tasks
  Ability to help troubleshoot remotely

Why is this appropriate for their needs:

The password-protected settings area addresses their needs perfectly it provides addresses their perfectly. It provides a "backdoor" to access any technical features. the activity logging feature (showing what threats were blocked) gives them peace of mind without requiring them to be there physically. They can also check remotely that the protection is working and that their relative is safe.

- The care provider (management)

Who they are:

The owner or manager of a home-care company, responsible for the overall direction of the organisation and for ensuring high quality care to all clients

Her needs:

- A solution that aligns with a care provider's values- the system must support client independence
- Evidence that the solution works – she need to see testing some user feedback and proof that the software actually solve the identified problem
- Clear documentation – she needs to understand what the system does and how it works, even though she is not very technical herself

Improves client wellbeing and independence

Reduce the need of staff spending time in computer rated support and also reduce cost as less money will be spent on carer training them how to fix technical difficulties

Her role:

- Provides access to information about clients' need and challenges
- Enabling contact with clients for research and testing
- Review the solution and providing key feedback
- Approving the final solution for deployment

Why is this appropriate for her needs and the organization:

The system is cost effective for the organization because it doesn't require on going subscription fee unlike commercial solutions.

From a strategic business perspective. it gives a care provider a unique selling point (a care provider that actively support client digital independence) which may attract more customer to the business. This could then differentiate them from competitors

- Care staff

Who they are:

Staff members at a home-care company who provide daily care and support. They are not IT specialist, but they often end up providing basic technical help as part of their general role

Their IT level:

Varies considerably – most of them are comfortable with modern technology but other aren't. most can handle basic task, but I wouldn't classify them as technical experts, because they receive training on care-related topics but typically no formal IT training

- **Their needs:**
  - Quick understanding of how it works – they need to be able to figure out how the system work without extensive training

    - Clear documentation – simple guides showing how to help clients with common tasks would be valuable for their training
    - Ability to explain the system to clients – the interface should be simple enough to explain clearly to clients.
- **Why is this appropriate for their needs:**
  The simplicity of the system makes it easy for care staff to support even without deep technical training and when something goes wrong, they will go straightforward to the built-in help features giving guidance. This self-documenting aspect of the design makes the system more supportable without requiring the care provider to invest heavily in technical training for care staff

SUPPORT USERS - software engineers

Who they are:

They are in-house IT professionals and developers working for a care provider. they are responsible for deploying the new application, integrating it with the organisation's existing IT infrastructure, and providing log-term maintenance and future updates.

Their IT level:

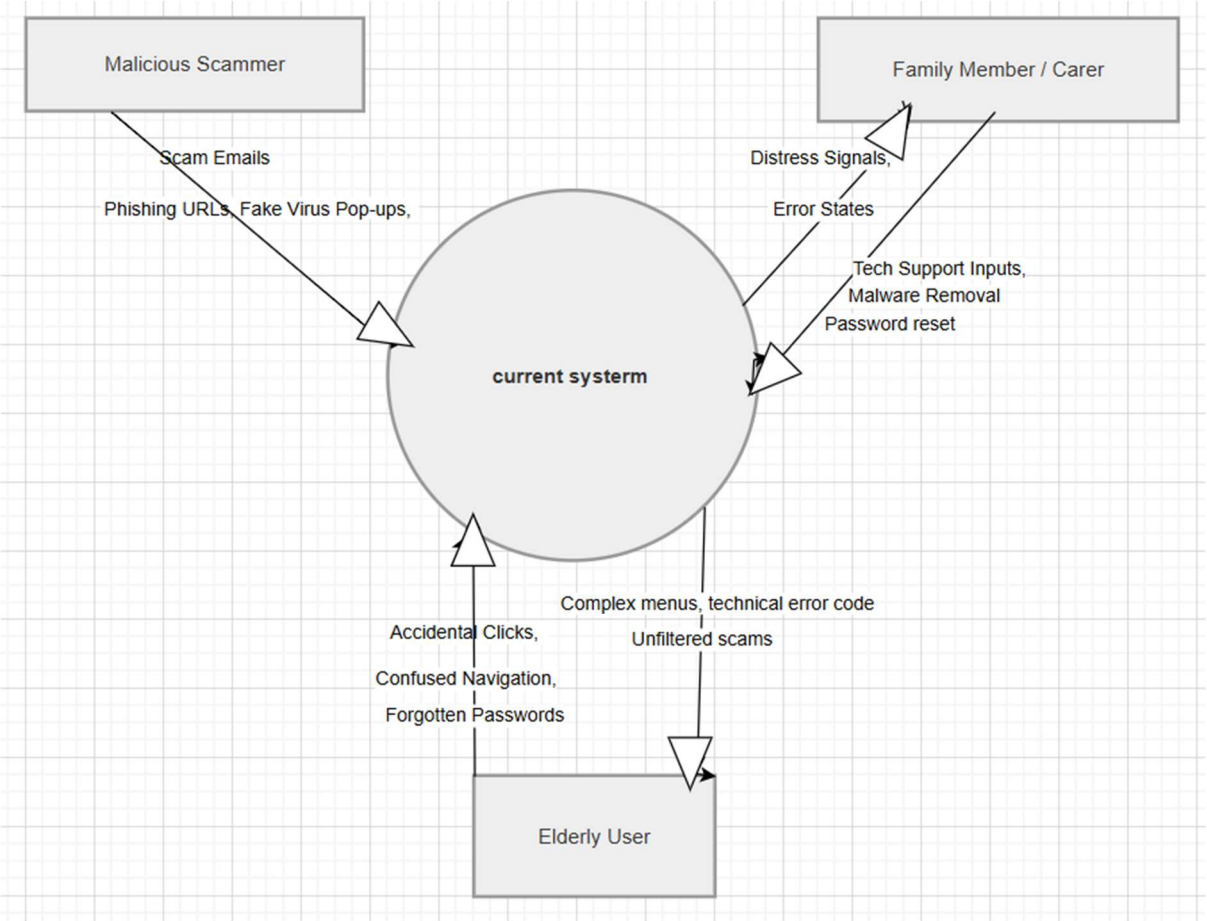
Expert. They are highly skilled in programming, system administration, and software deployments. They understand complex code and system architecture.

- Their needs:
    - Clean, well-structured code so they can easily read and understand how the system works
    - A secure way to push system updates, patches and new scam definitions to the app without disrupting the elderly clients
    - Access to system crash log or performance metrics so they can trouble shoot technical bug behind the scenes.

Why is this appropriate for their needs:

The modular design of the application addresses their needs perfectly. By using decomposition, the code split into specific modules(like the launcher, the scam detector, and the settings). This mean that if the engineers need to update the scam database, they can do straightforwardly without breaking the whole application. Furthermore, because the code will self-documenting with clear variables and comments, it makes the system highly maintainable. The app will also Output crash logs directly to them, giving them peace of mind and reducing the time they spend searching for bugs

DFD level 0- current system



- Explanation and justification of DFD

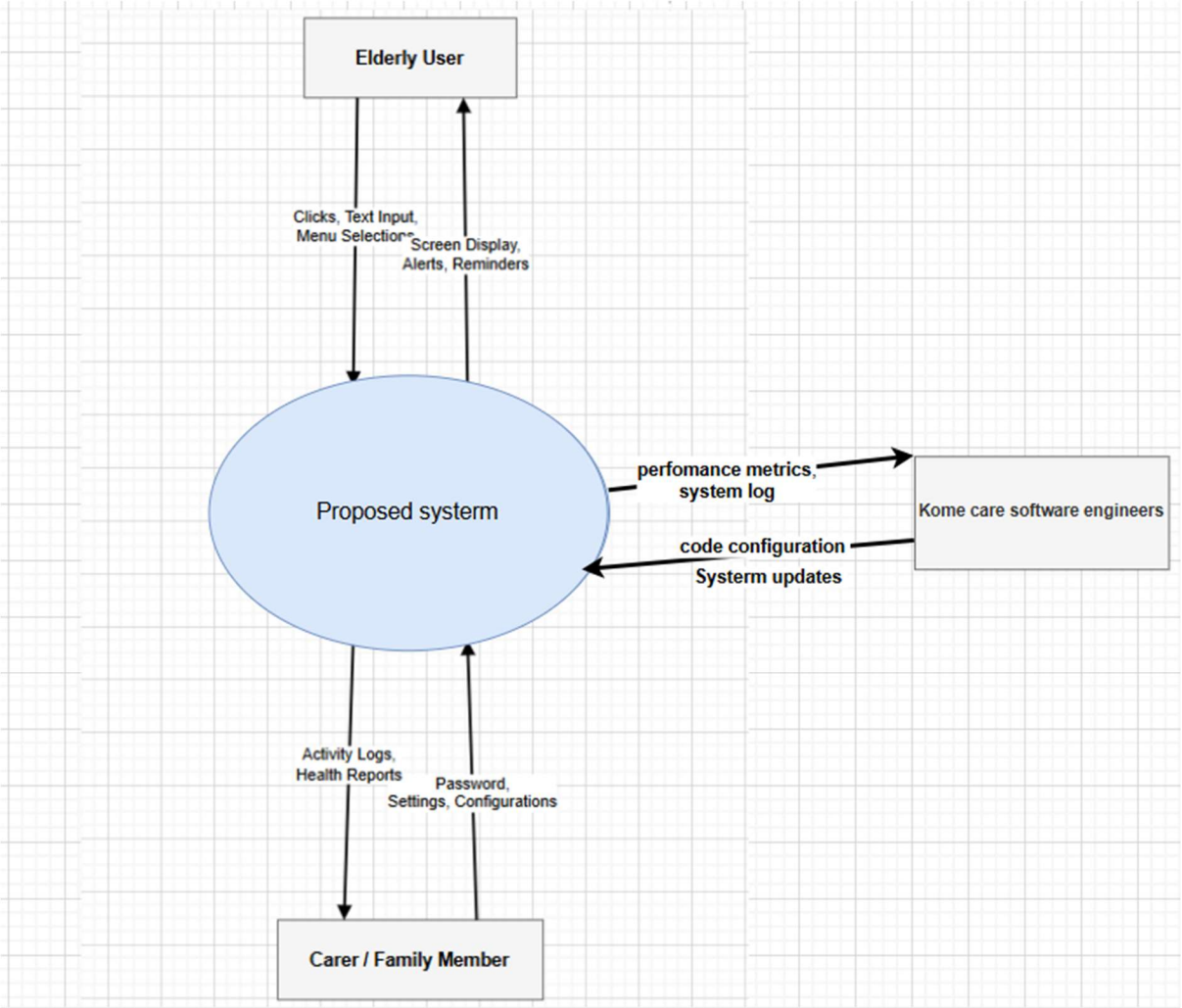
This DFD level 0 illustrate the current system that elderly users are experiencing. The diagram shows how the window desktop sits as an intermediary between users and various inputs/outputs however it is not an effective filter

The most concerning aspect shown in the diagram is the malicious content from the 'Malicious Scammer' entity. You can see that inputs like phishing URLs, fake virus pop-ups, scam emails flow through the system virtually unfiltered straight to the elderly user, because the current OS has only basic security features, it outputs complex menus and technical error codes alongside these unfiltered scams.

The diagram also illustrates the heavy dependency relationship with the family. Whenever the system outputs error states, the family member or carer has t constantly intervene. They are forced to input tech support, malware removal and password resets, making the current system quite complex and stressful for everyone involved

This DFD justifies the need for my proposed solution by visualizing the current problem in clear structure way . This means that it shows that the current system is not just inconvenient and it is actively failing to protect users and creating dependencies which might affect their well being

DFD level 0- proposed system



- Explanation and justification of DFD

This DFD level 0 illustrates the proposed system that elderly users will be experiencing. The diagram shows how the new integrated care desktop app act as a safe intermediary between the elderly users and their daily computer tasks, however, unlike the old window desktop, this system acts as a highly effective filter,

The most concerning aspects form the old system have been completely removed. The diagram shows how the app takes basic input like clicks , text input , and menu selection from the elderly user, and processes them securely to output a simplified screen display , helpful alert and reminder, because the app has a built in security features, it protect them by filtering out malicious content before it ever reaches the screen, making them much less vulnerable.

The diagram also illustrate healthier dependency relationship with the family. Instead of constantly having to do malware removal and fix errors, the carer or family member can simply input password, setting and configuration through a secure backdoor

Finally, the diagram introduces the software engineers. To ensure the system longevity the engineers can input system updates and threat database patches behind the scenes

The DFD Justifies my proposed solution by visualizing the new system in a clear structure way. This means that it shows that the proposed system is highly convenient, actively protecting users, and reducing stressful dependencies which greatly improve their well being and independence

Research into the problem

Before I started designing my solution, I needed to understand the problem from different perspective and gather evidence to support my idea. I used both primary research (data I collected myself) and secondary research (existing data to build a complete picture on how what it will take to build upon this idea

Primary research: I spoke informally with carers and older computer users about what they find difficult. This gave me a direct view of my target users and their needs. The downside is that it is time consuming and the number of people was small, so I have not presented it as statistics.

Secondary research: used published data from organization like action fraud and Age UK. This was quicker and showed that this problem extends beyond one care company and it is since as a national issue. However, the key thing I had to put in mind is that I have to be careful about using reputable sources since not all published research can be trusted or reliable or even up to date

So using both means I got specific user insight (what elderly users struggle with) backed up by broader evidence (that shows this affects elderly people nationwide)

- What people told me:

The same things kept coming up. There are too many icons and pop-ups on the screen. Warnings are written in technical language that people click through without understanding. People worry about scam emails pretending to be from banks, HMRC or delivery companies. Passwords get forgotten or written in a notebook. And people feel bad about always having to ask family or carers for help. These points shaped the requirements below.

- Secondary research:

To validate that these problem extend beyond the people I spoke to, I researched published statistics

Briefing

Facts and figures about digital inclusion and older people

July 2025

Digital Exclusion:

The office for national statistic report that approximately 2.7 million people aged 65+ in the UK have never used the internet, among those who use it. many lack confidence and struggle with basic tasks this digital exclusion affects access to healthcare, banking , shopping and social connection (Got it here at https://www.ageuk.org.uk/latest-press/articles/2024/more-than-1-in-3-over-65s-4.7-million-lack-the-basic-skills-to-use-the-internet-successfully/)

Scam impact:

According to Action fraud, people aged 60+ lost thousands of pounds to fraud in 2023, common scams targeting elderly people include phishing email appearing to be from banks or government agencies, fake tech support and some fraudulent investment scheme. And this lead to some psychological impact which often include depression, loss of confidence and complete avoidance of technology

Report a phishing attempt:



Cyber criminals use fake messages as bait to lure you into clicking on the links within their scam email or text message, or to give away sensitive information (such as bank details).

These messages may look like the real thing but are malicious. Once clicked, you may be sent to a dodgy website which could download viruses onto your computer, or steal your passwords **.(https://www.reportfraud.police.uk/how-to-report-suspicious-activity/)**

Product guidelines

RNIB
See differently

Accessibility guidelines:

The RNIB recommends minimum font sizes of 16px for elderly users and high contrast colour schemes. Research shows that larger click target (minimum 60x60 pixel) significantly improve usability for people with declining motor skills (https://media.rnib.org.uk/documents/Complete_Product_Design_Guide.pdf)

- Existing solutions research :

I researched current software to understand what has already existed and I could identify any gap my solutions could fill.

Simplified interface:

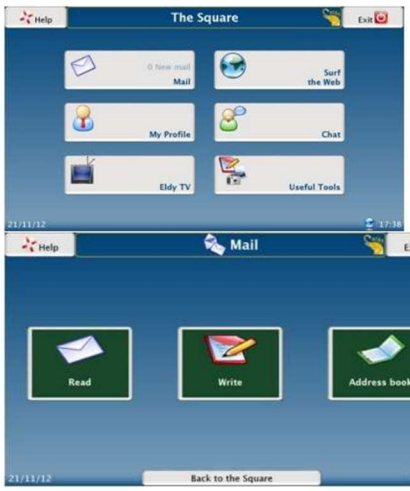


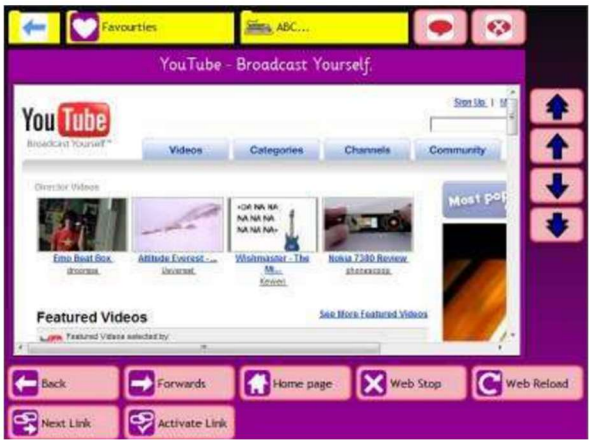
Eldy – this a free software with large buttons for basic functions

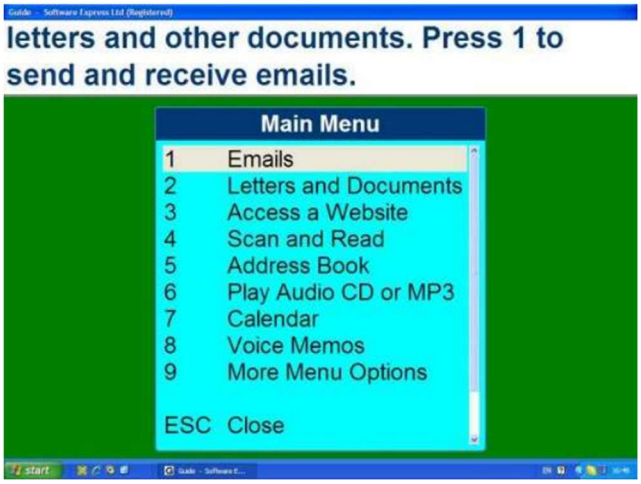
(http://www.eldy.eu/en/)

Eldy is software that turns any standard PC into an easy-to-use computer for people who have never used a computer before. It provides a simple interface that allows easy access to all the commonly used functions of a computer including email, internet, chat, documents, pictures and skype. It is free to download from www.eldy.eu and can be set up to start automatically and sit over the regular windows interface.

The home screen is called the “The Square” (pictured below left) and from here all features can be accessed within a couple of clicks.







- **Strength:** simple, free, large buttons
- **Weakness :** No security features, outdated (2018), doesn't protect from scams
- **Relevance:** proves simplified interfaces work, but missing critical security

Pointerware – simplified desktop with step-by-step instructions (£8/month)

PointerWare is a software that makes computers simple and easy for seniors to use. PointerWare is software designed with first-time elderly computer users in mind. You can configure PointerWare to start immediately when the computer is turned on, hiding the complexity of Windows with an elegantly simple interface.

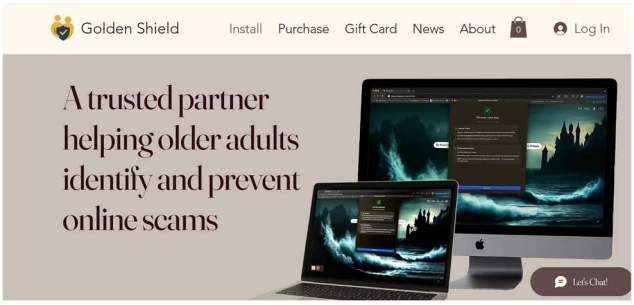
Designed with first-time computer users in mind, each PointerWare client gets their own email address. Family and friends can add pictures and contact information to the users visual address book. gotten from (https://pointerware.software.informer.com/)



- **Strength:** good guidance for low IT skills
- **Weaknesses:** No security, ongoing subscription cost
- **Useful:** Shows users will pay for simplification

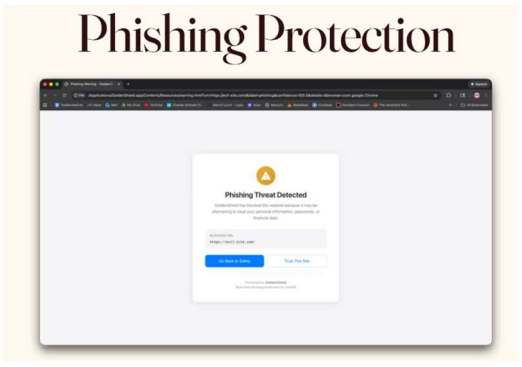
SECURITY TOOLS:

Golden shield: AI powered scam detection gotten from (https://www.goldenshieldprotection.com/)



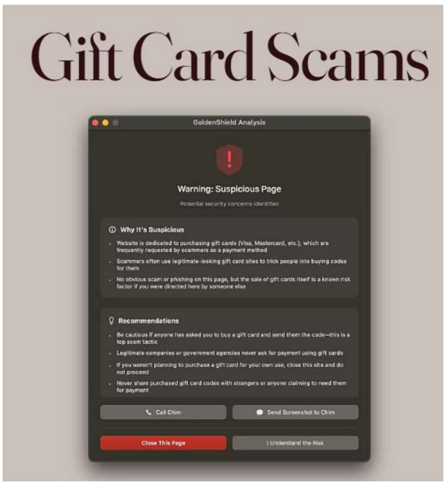
What is Golden Shield?

Golden Shield is a cybersecurity software that protects older adults from the most common and financially devastating online scams–gift card schemes, remote access applications, phishing emails, and more.

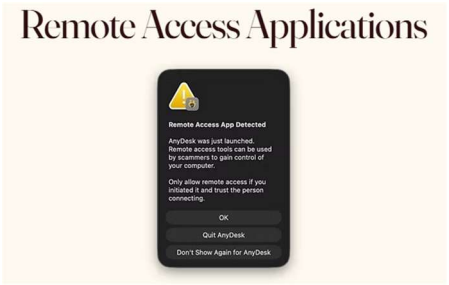


Empowered by AI, Golden Shield's software passively checks every website you visit against thousands of known phishing links without tracking your search history or slowing down your computer.

Our software will only trigger when you access a link known to be related with fraud.

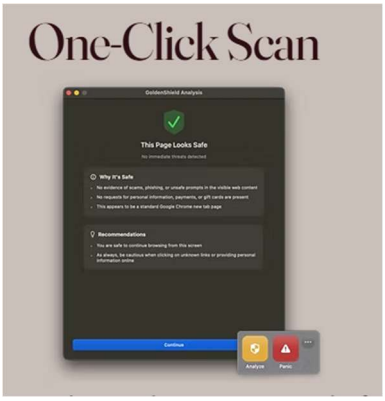


Scammers will often call seniors demanding urgent payment, directing them to purchase gift cards online to send them "money" through email. Golden Shield passively scans for gift card searches, politely warning against fraudulent requests for eGift Cards and suggesting they call a trusted person.



"Tech support" scams are the most financially devastating schemes. Fraudsters call seniors, announcing themselves as tech support, demanding they download a Remote Access Application–like AnyDesk–on their computer to access bank accounts.

We shield users from these frauds and warn against their sinister utilities.



Anytime users worry that something on their computer may be fraudulent, simply click "analyze," and our AI program will scan your screen for signs of fraud. It will examine everything visible: logos, URLs, email addresses, and more, to give you proper guidance and peace of mind.



While developing Golden Shield, we heard from seniors who had previously been targeted by a scam that they faced an active and immediate threat, but simply couldn't shut down their computer in time. No worry, with the "panic" button, all your browsers will shut down and your computer will restart automatically, eliminating threats.

Summary:

Strengths: good detection, real time protection

Weaknesses: complex interface, technical warning, not designed for elderly users

Useful: has needed security but wrong interface

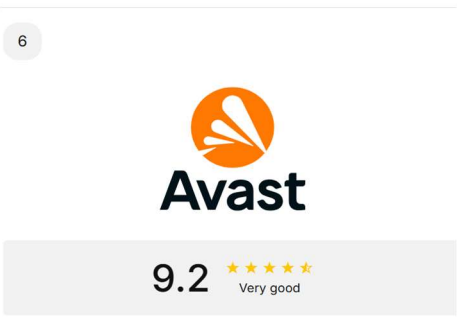
Standard Antivirus (Norton, McAfee) – comprehensive security suites

gotten from this website(https://uk.norton.com/promo/paid-search/norton-360)

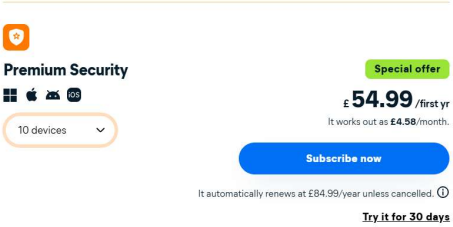
Award-winning antivirus now with AI-Powered Scam Protection.

Get powerful, easy-to-use, antivirus software that helps protect your devices against advanced online threats. All Norton 360 plans include AI-powered Scam Protection to keep you safer from scams.





Avast is a top choice of millions of users worldwide, providing comprehensive security solutions. It great at detecting viruses, not only on desktop, but also protecting mobile device from malware. gotten from (https://bi.cybernews.com/avastantivirus/)



Summary of this section:

- **Strength:** Effective detection
- **Weakness:** expensive(£30-55/ year) overwhelming interfaces, constant notification , and a lot of technical jargons
- **Useful: shows why current security doesn't work for elderly users (it might be difficult to navigate around the software, technical word are thrown to them and they might not even understand and they may get overwhelmed and quit. Or some might not even know about them)**

The critical Gap:

The critical gap is the no solution combined the simplified interface AND security in on integrated package design for elderly users . to get both today user need to install the simplified interface plus antivirus and also need help to configure them. This actually creates MORE complexity. Installing three separate programs is exactly that overwhelms elderly users.

Why my software will fill in this Gap:

My solution provides simplified interface (like Eldy) AND security protection (like Golden shield) in ONE program with consistent interface and simplified communication throughout

Furthermore this research tells me what to learn from and what to avoid. Adopt Eldy simplicity but add missing protection. Adopt Golden shield's security but communicate in simplified terms instead of technical jargon

CLIENT REQUIREMENTS:

Based on my research findings and conversations with people working in home care, I have identified exactly what my software must do and how it should be perform. These requirements are divided into functional requirements and non-functional requirements

- functional requirements:

these are the feature my software must include, derived directly from my research data

1. Simplified launcher interface
    - Must launch automatically when computer starts
    - Must run in Fullscreen, replacing the window desktop
    - Must display exactly large, clearly labelled buttons
    - Each button minimum 60x60 pixels(this was based on the RNIB guideline)
    - High contrast colour scheme for visibility
    - Must include shutdown button so users never to access windows

Justification: the people I spoke to struggled with too many icons and wanted a completely separate simplified interface. So the few menu button at the home section approach is justified

2. Check a message Scam detection tool
    - Text input for pasting emails/SMS message
    - Scans for phishing indicators:
        - Common keywords(“urgent”, verify account” suspended”, “click here now “
        - **Gift card requests(** inspired by golden shield common elderly scam where fake call/ emails request gift card payment)
        - Request for personal banking information
        - Fake urgency tactic
    - Check embedded URL against threat databases
    - Analyses sender addresses for spoofing
    - RED/AMBER/GREEN rating with immediately or “look legitimate”

Justification: older users need help identifying safe messages and are a common target for scams. Also from golden shield I also noted that elderly people are told to buy gift card and send coded thinking they are paying bills or helping family. This justification addresses how vulnerable they are.

3. My password Secure vault
    - Must store website name and password with encryption algorithm I will integrate
    - Must display password in simple list view with search function
    - Must include the password strength indicator
    - Password must be encrypted before storage
    - Must use master password for access
    - Also aid them in making password so it wouldn’t be difficult
    - May suggest to make a strong password for them and save it name to something they can reference from

    Justification: many older users struggle with remembering passwords. One person I spoke to mentioned she had to keep a full notebook of passwords, therefore a simplified manager built into the system is needed

4. Check a link scanner

- Accept direct URL
- perform instant safety checks: typo squatting detection (amaz0n.com vs amazon.com), domain age check, SSL certificate validation, comparison with phishing databases
- Must explain the verdict ("This URL is misspelled - the real Amazon is amazon.com")

Justification: Even with the safe browser user may need a way to check links before clicking on them from text messages. This is way to check links

5. My diary/reminder tool
    - Adding dated text
    - Support simple reminder creating for like (medication appointment and with description)
    - Must display entries in chronological view
    - Allow editing and deleting

Justification: user will need basic organizational tools. Rather than forcing them to use windows calendar or external app (which might me stressful to setup) including It in my software will reduce the need to exit the simplified environment

6. Setting & Accessibility
    - Adjusting font size
    - Include volume control
    - Provide language option (at minimum: English with framework for adding others)
7. Security & privacy features
    - Encrypt all password using library before storage
    - Hash master password
    - Maintain activity log of blocked threats for user review
    - Validate all user input to prevent code injection

Justification: data protection is critical. Elderly users handle sensitive financial and health information. The system must protect this data. Activity logging provide reassurance that protection is working without compromising privacy.

- Requirements comparison table

For key design decisions, I considered multiple approaches and selected the best option with justification:

Required feature	Option 1	Option2	Final choice	Justification
Login system	Text boxes for username/ password	Fingerprint/ face recognition	Text boxes	Test boxes don't require special hardware(accessible to all users regardless of device)elderly users are

				familiar with traditional password (they already use them for banking), simpler to implement, fingerprint readers can fail with skin condition common in elderly people, lower cost to deploy
Message scanning	Manual paste and check	Automatic email integration	Automatic email integration	While manual pasting is simpler to code my research showed the target users have a low IT skill level and struggle with navigating multiple windows to copy/paste text. Automatic integration connects directly to their inbox(e.g, via IMAP), scanning incoming messages silently in the background. T his removes the burden of manual operation, providing a seamless automated safety net that better supports elderly independence, justifying the higher development complexity
Password storage	Store in local text file	Store in SQLite database	SQLite database	More structure and reliable than text files, built in support for encryption industry standard for local data storage easier to search and retrieve specific password reduces

				risk of corruption, SQlite included with python(no extra dependencies)
Check message	Build own database of threats	Use google safe browsing API	Google safe browsing API	Google maintains database of millions of known threats (far beyond what I could compile), updated constantly (new threats added daily), free tier allows 10,000 lookup per day (more than sufficient) , industry -trusted source, let me focus on interface rather threat research
Setting persistence	Save to a cloud database	Save locally only(SQLite or JSON file)	Save locally	Elderly users may have unreliable internet connections, and cloud storage introduces unnecessary data privacy risk for vulnerable individuals. Saving settings locally ensures the application works offline and loads instantly. Most importantly, it allows the carer to configure the accessibility settings (like 24px font size) once, ensuring a seamless, “set- and -forget” experience for the elderly use without requiring them to reconfigure the app on every launch

Full screen mode	Standard minimizable window	True Fullscreen (cannot be closed at all)	Full screen with password protected exit	Many elderly users are overwhelmed by the standard OS . A standard window allow accidental minimization, leaving the vulnerable user stranded on the complex windows desktop. Conversely a completely inescapable Fullscreen prevents carers from performing necessary underlying system maintenance A password- protected exit perfectly balance the elderly user's need for a simplified foolproof environment with the carer's need for administrative access

Hardware requirements:

User' computer minimum specifications

- Operating system: windows 10 or 11
- Processor: Intel core i3 8th generation or equivalent AMD
- RAM: 4GB minimum
- Storage: 500MB free space for installation
- Display: 1366x768 minimum resolution
- Input: Mouse and keyboard
- Internet: Broadband connection for URL checking features

Justification: these are modest requirements that most computers from the last 5-6 years will meet. Elderly users typically have basic, budget computers purchased by family members. By keeping requirement low, I ensure software is accessible to all client, not just for these with newer hardware. Window 10 and 11 were chosen because despite window 10 recently reaching the end of its mainstream support lifecycle, it remain on of the common operating system on older budget laptop

used by the uk elderly demographic ensuring compatibility with older hardware is vital for preventing digital exclusion

Optional Hardware:

- Webcam/microphone (if users want to video calls the software could launch their existing video app securely).

Software Requirements

Development Environment:

- Programming language: python 3.10 or higher
- GUI framework: Tkinter(include with python) pyQt5
- Database: SQLite3(include with python)
- Additional libraries:
    - Request: for URL checking API calls
    - Cryptography – for AES-256 password encryption
    - Hashlib – for master password hashing

User's computer software setup:

- No additional software installation required beyond software.
- All dependencies will be bundled with the installer.
- Application packaged as a standalone .exe file

Third party services:

- **Google Safe browsing API** (or equivalent) for URL checking
- Free tier allows up to 10,000 API calls per day API keys.

Limitation of the proposed solution

While my proposed computation solution directly addresses the core problems identified in my research, there are inherent limitation to the system that must be acknowledged prior to development.

Limitation	Why is this is a limitation
Connection dependency	A major feature of this app is the scam detection engine which queries URLs against the Google safe browsing API. This is strictly require an active internet connection. If the elderly user's Wi-Fi drops will not be able to verify new links, temporarily limiting its effectiveness to just the offline string heuristics.
False positive and negatives (software accuracy)	The scam detection algorithm will rely on keyword scanning (e.g searching for

	"urgent", "account suspended). A limitation of this algorithmic approach will be it may occasionally flag a legitimate, urgent email from a real bank as a scam(a false positive). In contrast, a highly sophisticated phishing attack that avoid the common keyword might slip through unflagged (a false negative)
Hardware constraints on older devices	My app requires overriding(kiosk mode) the windows desktop and running python/Tkinter locally while I have optimised the requirement(4GB RAM), many elderly users run extremely outdated machines with degraded hard drives. This could affect the goal of the app launching within 5 seconds, causing slight delays that might confuse the users.
Human Factor (user ICT skills)	Even though the app uses a highly simplified button interface with severe cognitive decline or an IT skill level of 0/10 will struggle to operate it without supervison furthermore, if users physically write down the carer backdoor master password on a sticky note by their desk, it could lead to human ee
Check message feature	Apart from the automatic check the manual check where the elder could type in a message and validate if it is true this feature will relies on the user manually typing SMS text messages from their phone or other places into there phone into the application. As their system run on a desktop computer, it cannot directly access messages from a separate mobile device

SUCCESS CRITERIA

Success criteria are vital to this project as they give a clear measurable understanding of what need to be done to achieve the best possible outcome for elderly users and their carers . setting these criteria now allows me to test the application continuously during development and forms the entire basis of my final evaluation. To ensure my success criteria are measurable

- **Quantitative** : this is involves using numbers and hard data to measure success. For example, measuring the exact time the app takes to load , verifying the pixel dimension of UI elements,

or calculating the percentage accuracy of the scam detector using a test batch of known phishing links

- **Qualitative** : this involves concepts that cannot be measure with raw numbers such as confidence or readability. I will measure this by gathering direct feedback from the elderly via one ot one interview to see if their experience improved

Measurable success criteria list

Based on my analysis and the client requirements, my final product will be evaluated against the following criteria

Quantitative criteria:

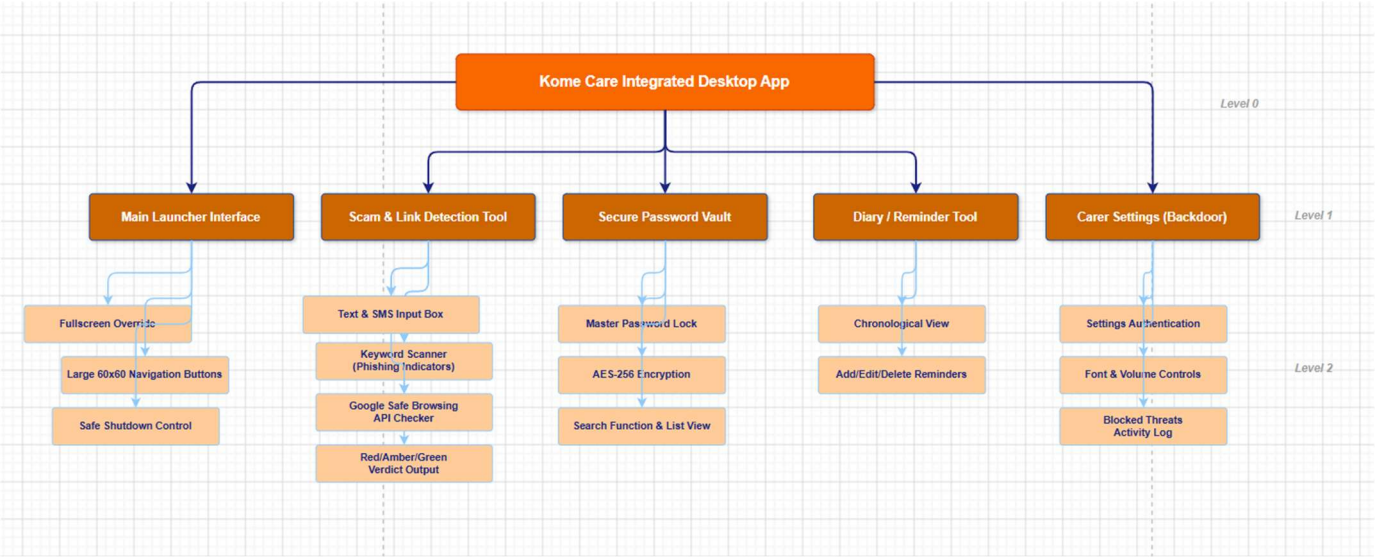
- **SC1- Performance:** the software must launch and completely override the windows desktop within 5 seconds of execution on a standard 4GB RAM test machine
- **SC2- Accuracy:** The scam scanner algorithm must correctly process and flag at least 95%of a controlled test batch of known phishing text strings and URLs.
- **SC3 - Accessibility:** 100% of the clickable interaction button must meet the RNIB minimum size requirement of 60x60 pixels.
- **SC4 - Security:** Password stored in the secure vault must be 100% encrypted or hashed before being written to the local SQLite database
- **SC5 - Authentication:** The system must successfully block 100% of unauthorized access attempts to the carer backdoor menu when tested with incorrect master password

Qualitative criteria:

- **SC6 - Client satisfaction:** During post-development users testing a sample group of elderly users (aged 65+) must report an average ease-of use and confidence score of at least8/10 on a feedback survey
- **SC7 - Communication: Carers and non-technical testers must confirm through testing that all error message and scam alerts are are written entirely in plain English and no technical jargons**
- **SC8 - Usability:** an elderly user with a self-reported IT skill level of 2/10 msut be able to navigate the system without further assistance

DESIGN

Breaking the problem down:



I have chose to use decomposition to break down the overall KomeCare software into a series of smaller, manageable sub- problems . because this system need to replace the standard window desktop with a simplified interface while simultaneously running complex security checks in the background, developing it as one continuous script would be highly inefficient and prone to errors.

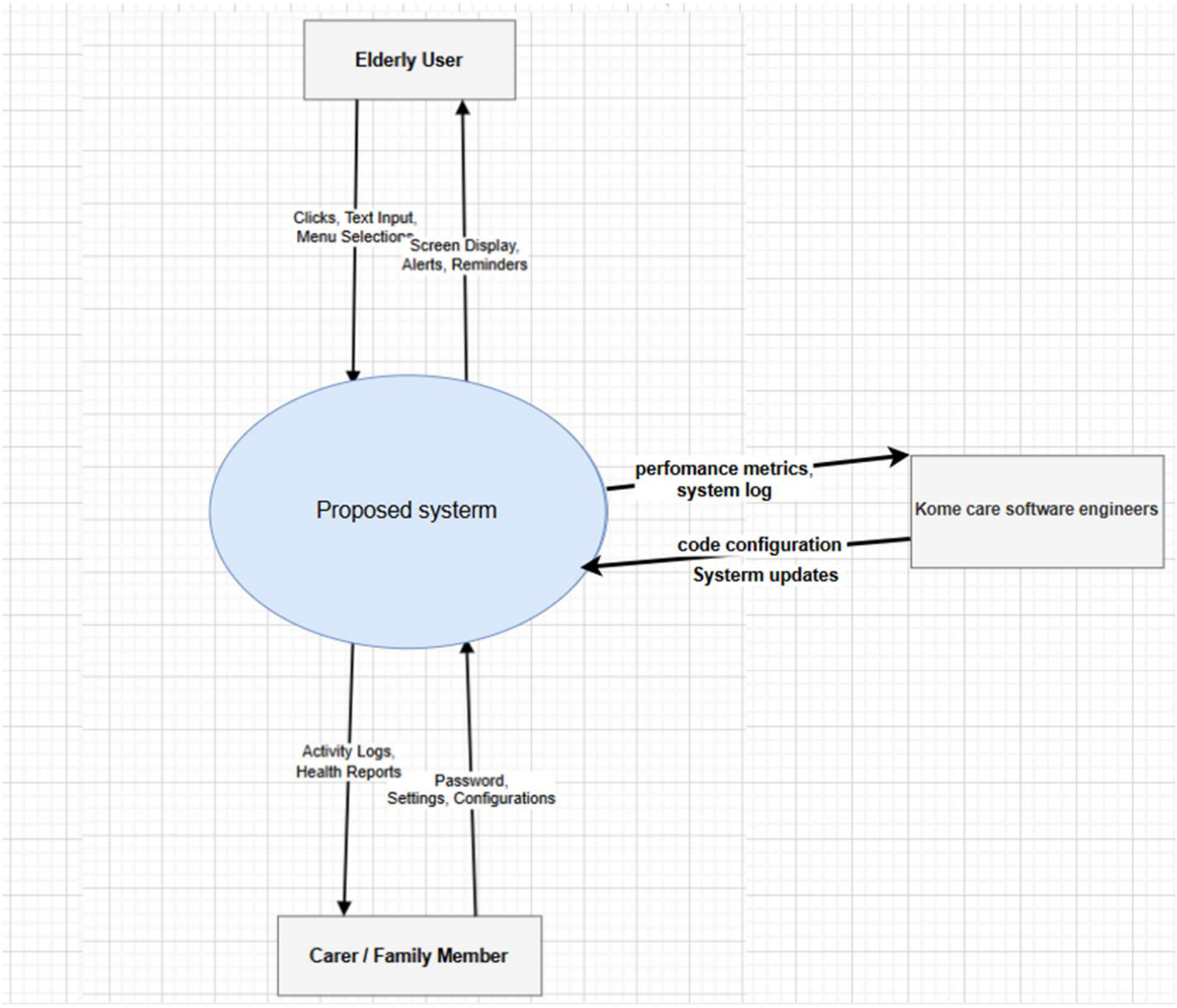
By structuring my solution into a hierarchy diagram, I have visualised the system as a step by step objective. This modular approach is highly suitable for this computational solution because It allows me to program, test and debug on specific component at a time. For example, if a bug occurs in the google safe browsing API, this modular structure ensures it will only affect the detection tool and will not crash the main launcher, allowing the elderly user to safely continue using their diary or daily applications

- **Main launcher interface:** This is a core graphical user interface that elderly client will interact within and will act as a kiosk mode that stays on top of z-order. This ensures the elderly user cannot “alt tab” by mistake from trying random keyboard keys this enable them not getting lost. The launcher will act as central controller as it will manage the lifecycle of the program and it most also meet the RNIB accessibility guidelines(the larger 60X60 pixel navigation buttons and also use will be it act as a return to home for example if the internet browser is open but hasn’t been touched for 5 minutes, the launcher can automatically bring the simple interface back to the front
- **Scam and link detection tool:** This module handle the primary security requirements. It uses a Heuristic analysis algorithm. Instead of looking for a single word , it scans for strings and patterns.it also uses an API request to check URLs against the Google safe browsing database.

Am implementing a weighted scoring system, each suspicious element will add to the “RISK SCORE”. If the score exceed a certain threshold, the amber/red alert is triggered. This is much more accurate.

- **Secure password vault:**
  This module handles data privacy. It contain the logic for the master password lock, it uses an SQLite database to store credentials. To keep it secure, tit uses implement a library that will work similar to AES-256 Encryption for the stored data and a hashing Algorithm for the master password.
  I will be implementing slated Hashing. By adding a random string to the password before hashing it , you ensure that even if two users have the same password , their stored hashes look completely and protect against unique attacks.
- **Dairy / reminder tool :** This handle information persistence. It allows the user to perform CRUD operation (which is create, read, update, delete) on their daily schedule. The tools uses chronological sorting logic. When the app pulls data from the SQLite database. For example it could use a SQL command on a Datetime. This ensures the user always sees the most urgent task at the top of their list without having to manually sort it.
- **Carer settings (backdoor):** This is the administrative configuration module. It is hidden from the main UI and acts a “Gatekeeper” for system-level settings like font size, volume, and security logs. By keeping this isolated, I guaranteed that an elderly user cannot accidentally alter their accessibility setting or break the system configuration.

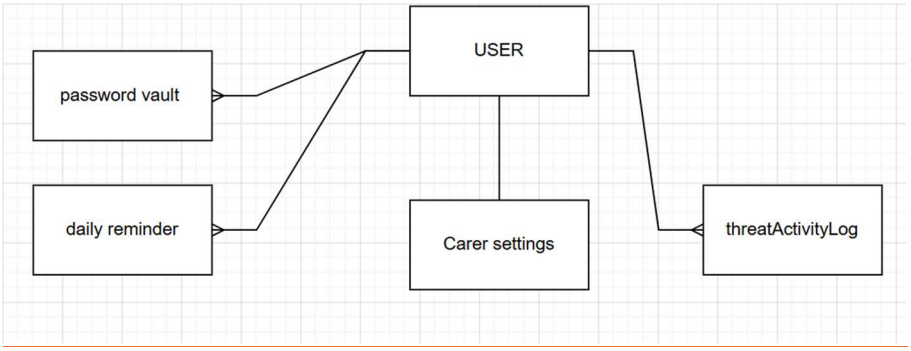
DFD level 0- proposed system



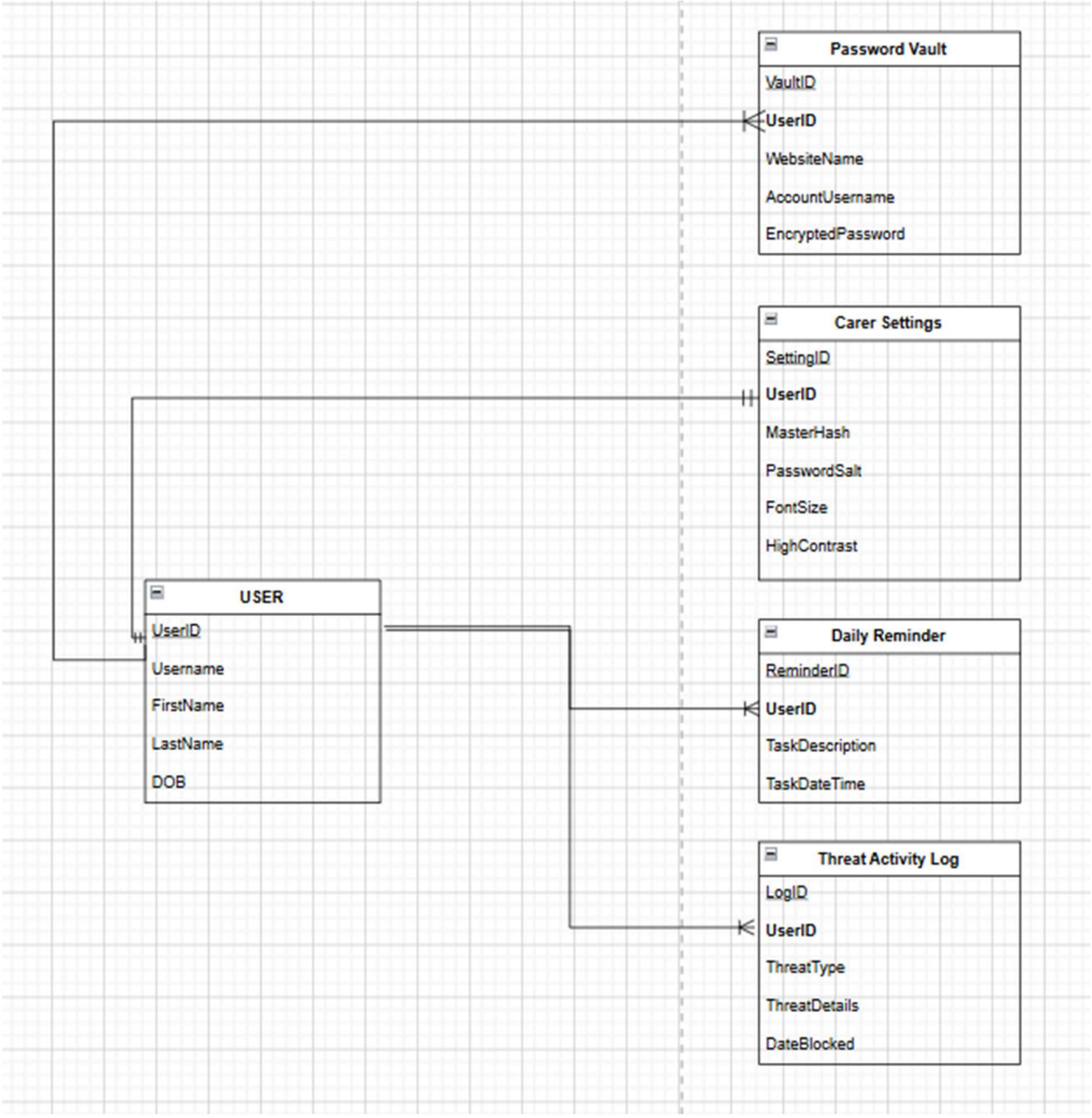
(ALREADY EXPLAINED AND JUSTIFIED IN THE ANALYSIS SECTION)

DATABASE AND ENTITY RELATIONSHIP DIAGRAMS:

The system I will need a database in order to store all the necessary information about my users, below are entity relationships diagram showing how all the data in the database relate with each other:



This diagram above shows the use of abstraction and focus on the main group of data that needs to be stored, now this uses abstraction is vital as it allows me to gain a better view of what stuff are needed to be included within the project , this will then allow me to create a more detailed entity relationship diagram:

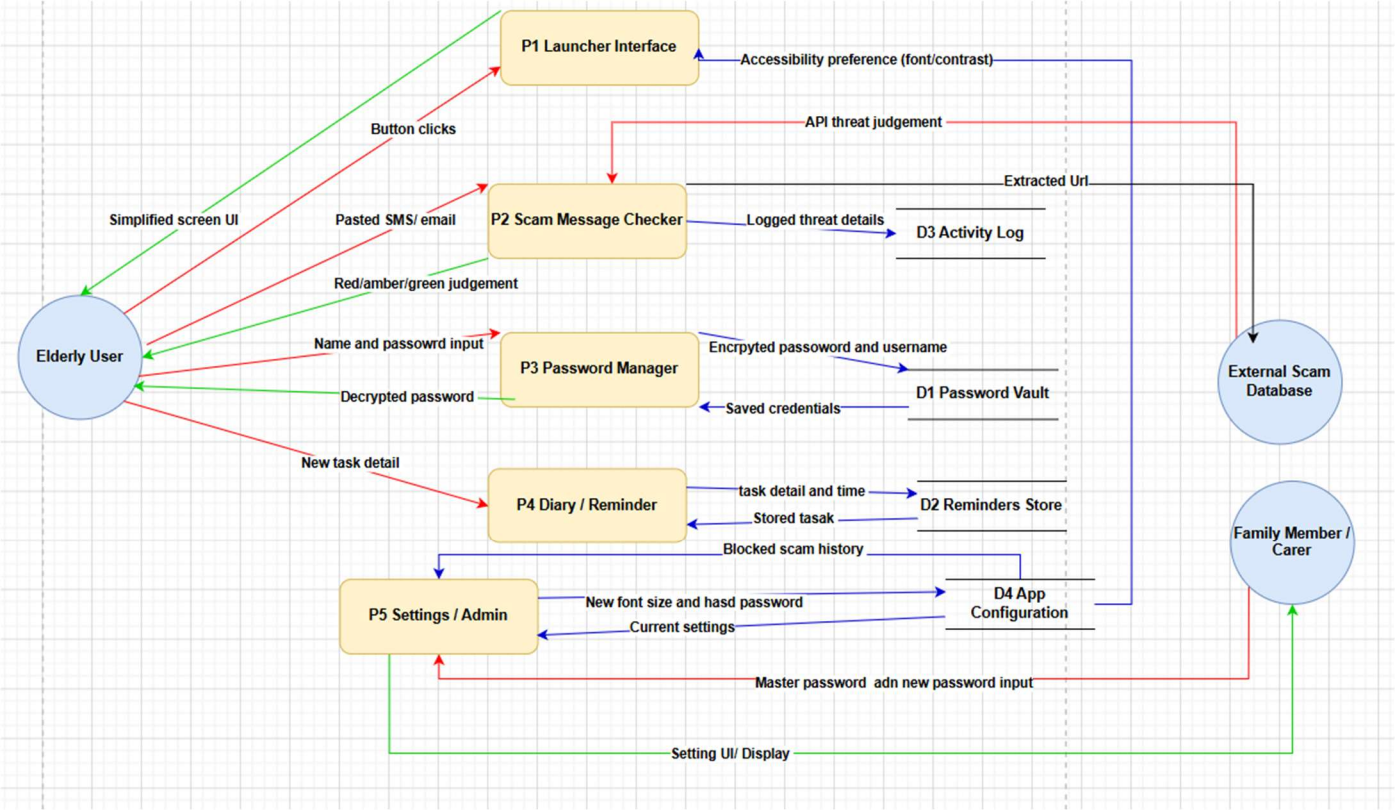


<u>Underlined</u> = Primary key

Bold = Foreign key

The diagram above shows the detailed entity relationship diagram for my proposed system displaying all the exact attributes and key required. This database is structured in third Normal form (3NF).each table has it own unique primary key, and all non-key attributes depends only on that given primary key.

Designing the database in this 3NF is important because it ensures data consistency and removes data redundancy. By linking the central USER table to the peripheral tables using UserID as a foreign key, the system maintains strict referential integrity. Aa shown using crow's foot notation, there is a one-to-one relationship for the carer settings, and one-to-many relationships for the vault, reminders, logs, whitelist. This clear structure allows for easy maintenance, meaning any future changes or scaling (such as adding a second user profile) would be a straightforward process.



Keys:

(——→) Red arrows: System inputs (Data entering from external entities)

(——→) Green arrows: System Outputs(Data displayed or sent externally)

(——→) Blue arrows: Internal Data transfers & storage

The level 1 Data flow diagram illustrate the exact movement, transformation, and storage of information within the KomeCare integrated software . by mapping the data flow, I can trace how raw inputs are securely processed into meaningful outputs, ensuring that the system logic aligns with the requirements .The data flow demonstrate that the primary elderly user interact exclusively with the simplified frontend processes(P1 to P4) conversely, the family member/ carer is structurally separated to the P5 settings/Admin module. This guarantees that

an elderly user cannot accidentally send data to, or modify, the D4 App configuration store, drastically reducing the chance of the system failing through manure errors.

Furthermore, I included coloured arrows showing the direction data flow and also the labelled arrow make it possible to visually see what exactly the major stakeholder are interacting with and why they are . designing the flow in this manner ensures the system is resilient, secure and actively protect the vulnerable end-users.

Data Dictionaries and Validation:

I will need to create data dictionaries as they will clearly outline the characteristics, attributes, validation rules of the data types that will be used within the proposed system. Each table below included a brief description of the field and justifies the validation method used to ensure data integrity. Implementing robust validation(such as presence, length, format, and range checks) is critical to prevent code injection and system crashes, which is especially important to maintain a frustration free experience for my elderly end users

USER: Description – to hold the primary information and identification for the elderly clients

Field name	Data type	Validation Rules	Example data	Justification
User ID Primary key	Integer	Type check (integer). **uniqueness check**	1	**Type check** ensure the generated ID is a number. **Uniqueness check** ensures the ID is a primary key so the system can accurately pull records belonging only to that specific user
Username	String	Presence check. uniqueness check. **length check** (max 20)	margaret1960	**Uniqueness check** prevent two profiles profile from conflicting **length check** ensures the strings does not exceed 20 characters to

				prevent wasting storage data and protect against buffer overflow
FirstName	String	Presence check Length check (max 30)	Margaret	Ensures the user's name is entered so the UI can provide personalised greetings. This is restricted to 30 characters to prevent UI overlapping on the screen
Last_name	String	Presence check . length check (max 30)	Abdullahi	**Presence check** to ensure the user has entered their surname, which is required for family members or carers to identify the correct account.
DOB	Data	Presence check. format check (DD/MM/YY)	11/11/1960	**Format check** makes sure the user enters their date of birth in the exact 10-character structure. This standardize the database and prevents formatting errors.

- SETTINGS:

Description: stores the carer's administrative credentials and the global accessibility preferences for the application UI.

Field name	Data type	Validation Rules	Example Data	Justification

SettingID (primary key)	Integer	Uniqueness check	1	Acts as the primary key to uniquely identify the settings profile
UserId(foreign key)	Integer	Referential integrity check	1	Acts as the foreign key to link these settings to the specific elderly user
MasterHash	String	Presence check. length check (Exactly 67)	‘E3b0chgy536735hgj…’	Stores the SHA-256 (Secure Hash Algorithm 256 bit) (hash of the carers backdoor password. **Length check** ensure it is exactly 67 characters. Storing hashes instead of plain text prevent unauthorized access if ever the local database is compromised
PasswordSalt	String	Presence check. Length check	‘jbsdHVDSIBSDVLYIV345332’	A random string appended to the password before hashing. **Presence check** ensures this is generated to prevent rainbow table attacks (which is a password-cracking method that uses a preplanned table to reverse cryptographic hashes and quickly uncover plaintext password)

FontSize	Integer	Presence check. Range check (16 to 48)	24	It uses dynamic scaling so instead of a hardcoded font size like size 20, the code will likely calculate the font size based on the window size . It ensure it does shrink below the RNIB minimum guideline (16px or grow so large it break the UI layout)(48px)
HighContrast	Boolean	Type check (Boolean)	True	Allows the carer to toggle a high visibility colour scheme. **Type check** ensures it is only True/False, which is the most memory efficient data for this simple on/off

- VAULT:

Description: store the AES-256 **(Advanced Encryption Standard).** encrypted login credentials for the elderly user, acting as secure password manager.

Field name	Data type	Validation	Example Data	Justification
Vault ID (primary key)	Integer	Uniqueness check	89	Uniquely identifies each saved password entry so the User can accurately update or delete specific records.
UserID (foreign key)	Integer	Referential integrity check	1	Foreign key linking the saved password to the correct user profile.
WebsiteName	Strings	Presence check. Length check (Max 50)	'NHS portal'	**Presence check** so blank records aren't saved. A length limit of 50 characters prevent UI overflow when rendering the list of saved websites on the simplified screen
AccUsername	String	Presence check Length check(Max 100)	'margaret.jones@example.com'	Stored as plain text so the elderly user can identify which account it is. Max 100 character accommodates long email addresses.
Encrypted Pass	String	Presence check length check (Max 255)	'USJSIBjkbabbjb232+...A=='	Stores the cipher (which is specific set of algorithm used to perform encryption or decryption) **Presence check** data is written. It is never stored in

				plain this is to protect the client against local malware or identity theft

- REMINDER:

Description: stores chronological task and appointments to support the user's daily independence.

Field name	Data Type	Validation rule	Example data	Justification
ReminderID (primary key)	Integer	Uniqueness check	7	Uniquely identifies the reminder so it can be targeted for a CRUD 'Delete' or 'Update' operation.
UserID (foreign key)	Integer	Referential integrity check	1	Foreign key linking the reminder to the main end user profile
TaskDescription	String	Presence check. Length check(Max 150)	'Take medication'	**Length check** is restricted to 150 character to force concise reminder that are easy for elderly users to read it quickly without the need of scrolling
TaskDateTime	String	Presence check format check (YYYY-MM-DD HH::MM).	2026-04-20 09:00	Stored as a formatted string to allow SQlite to execute chronological sorting. Format check ensures that the data are structured correctly so the app doesn't crash during the rendering

- ACTIVITY LOG:

Description: A historical record of scams blocked by the system for the carer to review.

Field Name	Data type	Validation Rule	Example data	Justification
LogID(primary key)	integer	Uniqueness check	**976**	Uniquely identifies the specific threat logged by the system
UserID (foreign key)	**Integer**	Referential integrity check	**1**	Links the blocked threat to the specific user being protected.
Threattype	**Strings**	Look up check	'Malicious Link'	**Lookup check** ensures that only predefined categories of threat are saved, which standardizes the logs and makes them easier for the carer to read
ThreatDetails	**String**	Length check(Max 200)	**'http://fakepaypal-nhs.com'**	String the exact string or URL that was blocked so the carer can review it **Length check prevent** the database from storing excessively long spam payloads
DateBlocked	**Strings**	**Format check (YYYY-MM-DD HH:MM)**	**'2026-04-19 17:30'**	It will be automatically generated by the system when a threat is blocked. Format check ensures constancy for chronological sorting.

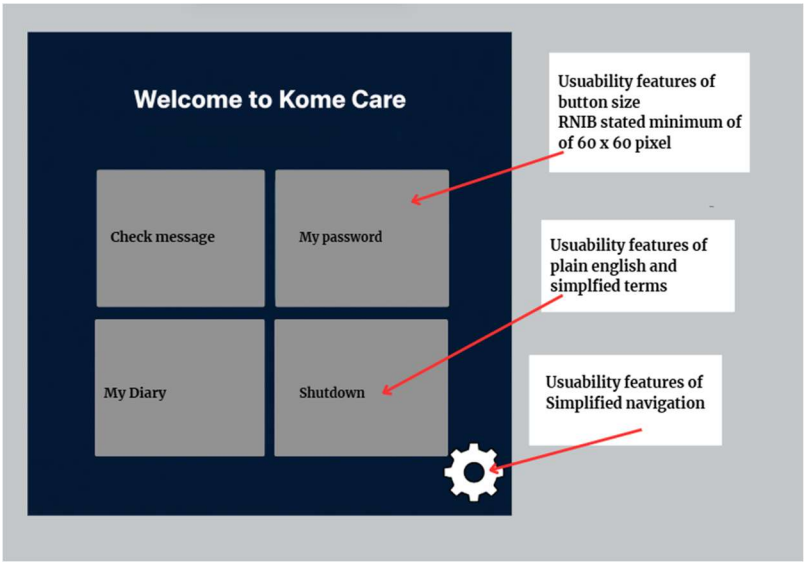
Usability Features:

As identified in my analysis, the target users have low IT skills and find modern computer interfaces frustrating. Because the target demographic is aged 65 and above, the system cannot rely on normal desktop transmission. To ensure the application is accessible, intuitive, and promote independence, I will be implementing the following specific usability features:

- Fullscreen 'Kiosk Mode' Override (Error prevention)
  The application will operate in a forced Fullscreen mode, entirely replacing the standard windows desktop and hiding the taskbar.
    - Justification: is that my primary research indicated that elderly user frequently click the wrong thing, accidentally minimize windows, and they get lost in the operating system leading to anxiety. By removing access to the underlying OS, this feature prevents fatal navigation errors and creates a safe, inescapable environment.
- RNIB-Compliant interaction Elements(Motor Accessibility)
  All clickable buttons on the interface will be strictly designed to a minimum size of 60x60pixels.
    - Justification: sticking to the Royal National institute of blind people (RNIB) guidelines discovered during my secondary research, large interactions target accommodate users with declining physical abilities or hand tremors. This makes the interface physically easier to navigate compared to standard, small desktop icons.
- Adjustable Typography and high contrast
  The system will feature a high-contrast colour scheme by default and will utilize scalable font size (ranging from 16px to 48px) which can be adjusted in the carer setting.
    - Justification: many elderly clients suffer from degrading eyesight. Allowing carers to scale the text and and ensure the system remains completely readable for all users, directly addressing the accessibility requirements identified in my analysis.
- Plain English and visual feedback (cognitive Ease)
  Technical jargon will be entirely removed from the system. Alert such as scam detection judgement, will use a simple Red/Amber/Green colour-coded traffic light system paired with plain English explanations (e.g "Danger : do not click this link")
    - Justification: my research showed that users are often overwhelmed by technical jargons / errors message therefore providing clear, visually intuitive , and colour coded feedback ensures users instantly understand the system output without needing any prior cyber security knowledge
- Simplified 'Hub and spoke' navigation
  The interface will use a remarkably flat navigation structure. Every sub tool such password vault, reminders, scam scanners will feature a single, prominent return to main menu button placed consistently everywhere in the same location on every screen
    - Justification: By avoiding deep, nested menus, the cognitive load on the user is drastically reduced this is useful as they will never be more than on click wa y from the

familiar home screen, making the system highly intuitive and preventing them form ever feeling ‘stuck’’’

Human computer interface:

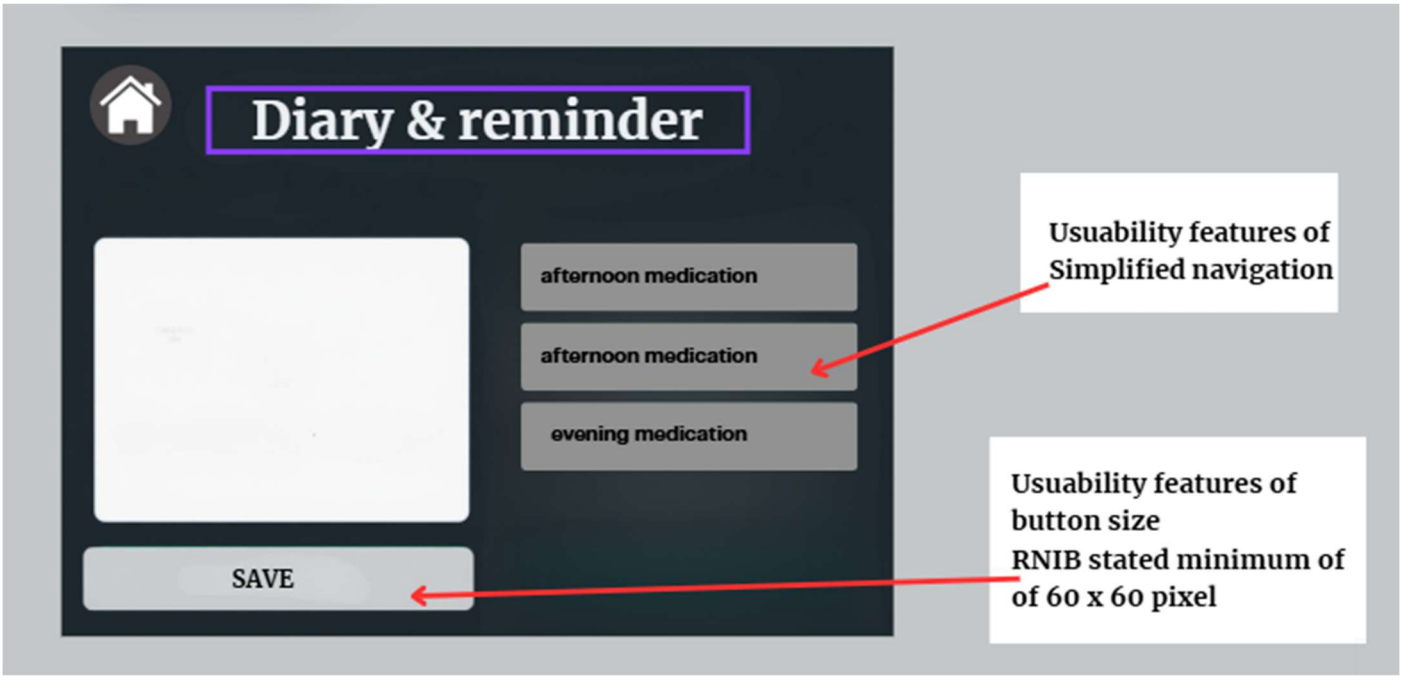


Screen one:

This interface acts as a centre kiosk for the KomeCare software

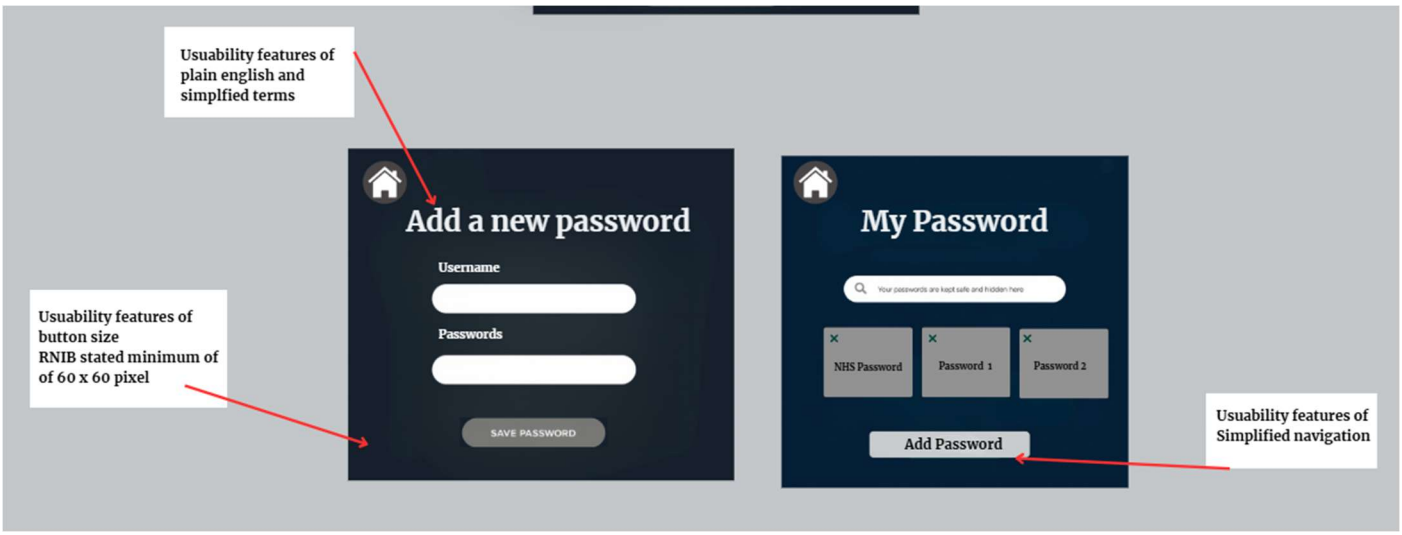
Usability features and justification:

- RNIB complaint buttons: The interface uses a grid 2 x 2 grid with massive interaction target and enables the size is more than the minimum 60X 60 pixel. This is good as it will accommodate users declining motor control and preventing frustrating mis clicks
- Uses of plain English all the technical jargon has been replaced with simplified conversational terms (e.g check messages)
- Restricted navigation the carer settings back door (the cog icon) is pushed to the bottom corner without the text label this is kept away to avoid misclick and confusion



This screen handles the data Persistence, allowing the user to seamlessly perform CRUD (create and read) operation on their daily schedule.

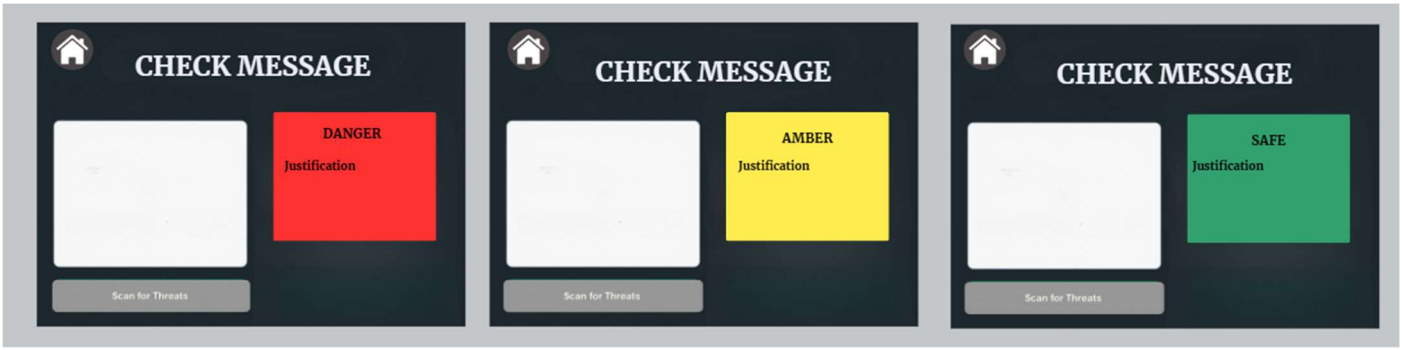
- Clear Data insertion: The oversized white input box and massive “SAVE” button (exceeding RNIB 60x60 pixel this ensures that typing and submitting a new task is physically accessible and straightforward
- Simplified navigation : reminder are displayed as clear distinct blocks on the right . by relying on the SQlite database to automatically sort these chronologically, the system provides “simplified navigation”. The user sees their next task immediately without having to figure out to use complex navigation and make ways through tabs and apps before reaching there
- Also added a universal known home icon to go back home so it will easy for them to navigate back home screen (I will place this on same position on every screen so it will easier to find them)



This interface acts as the frontend for the AES-256 encrypted passwords manager.

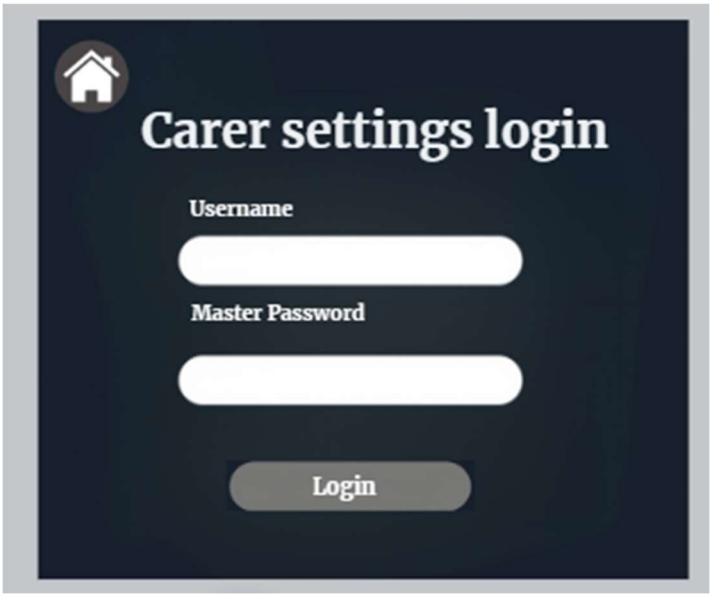
Usability features and justification:

- Plain English and clear label: the use of straight ward, conversation heading “Add a new password” and clear label of inbox boxes strips away the confusing cybersecurity jargon.
- Simplified navigation : retrieved password are displayed as large, distinct cards. The “Add Password” button ensures the user can seamlessly navigate to the data insertion from without hunting down through hidden menus
- Also added a universal known home icon to go back home so it will easy for them to navigate back home screen (I will place this on same position on every screen so it will easier to find them)
- RNIB: both the text entry field and the “save passoword” button utilize massive. This prevent physical frustration when typing sensitive data and ensure users with declining motor skills can confidently submit their credentials to the encrypted SQLite database.



This screen serves as the critical fronted interface for the heuristic scam detection algorithm and google safe API. This design protize immediate visual feedback, and the different type of result screen it will show

- Traffic light heuristic output complex algorithmic processing and JSON API responses are abstracted entirely away from the user. Which means instead the system simplified and output a universally understood Red(danger), Amber(warning), or Green(Safe) visual block. This instantly alert the elderly user to the threat level without requiring them to interpret cybersecurity data.
- Plain justification: alongside the colour coded warning given, the output provides a plain English justification, this will educates the user and provides actionable feedback, reducing anxiety and confusion.
- Unified input/output interface : by replacing the manual data insertion field (the white box text box) and the visual output directly next to each other on the same screen, the system eliminates context switching . The user does not have to navigate to a new page to see their result ensuring them a seamless and frustration free experience.



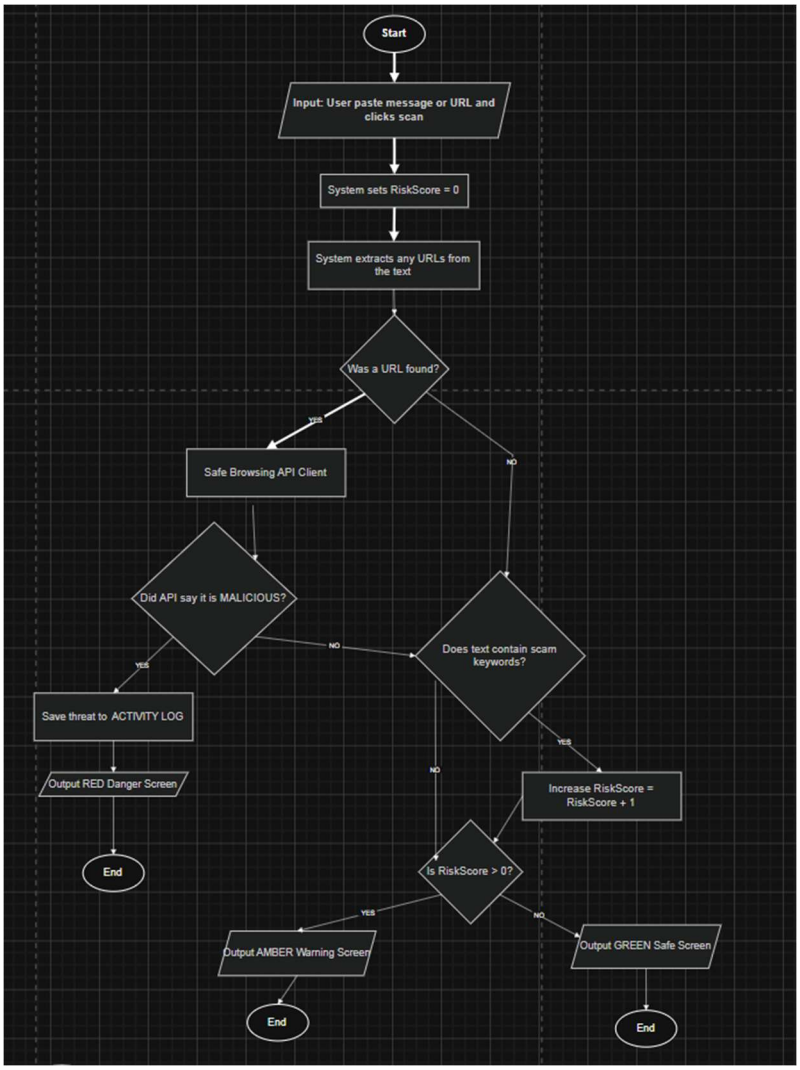
This interface acts as strict gatekeeper between the elderly user simplified environment

And the administrative system configuration. The design focus entirely on system authentication:

- Distraction free modal layout: the screen is intentionally minimalist. By removing all other navigation options, and it also give the elderly a visual clue they have a restricted area, preventing them from accidentally altering system settings.
- Securing the data entry point: this screen serves as the directed input for the system validation. The master password entered here will be processed by the python Haslib library, hashed via SHA-256 and validated against the stored credential in the SQlite TBL_SETTINGS databases before granting access.

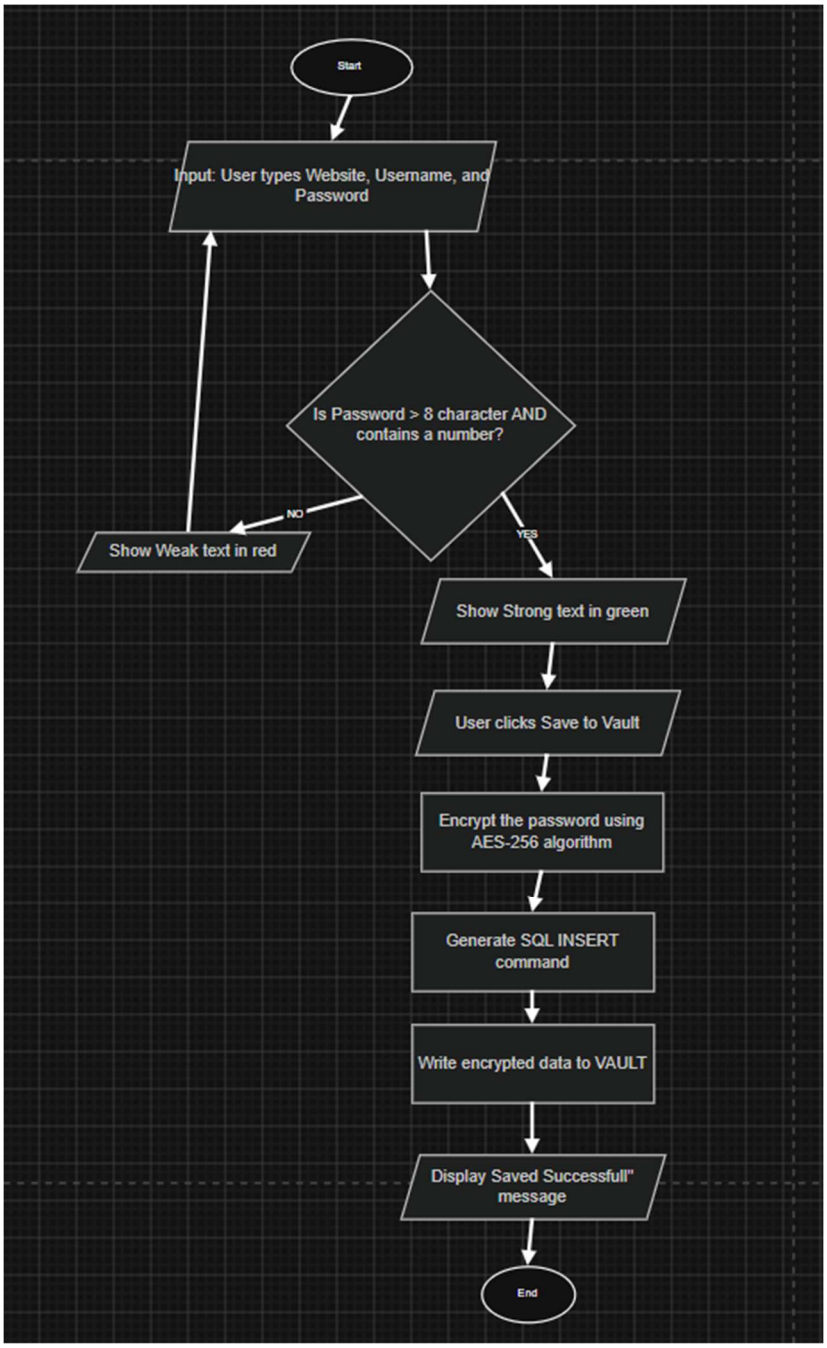
Algorithms:

- Threat scanner flowchart:



The flowchart above illustrate the branching logic of the heuristic scam scanner, this demonstrates how the system default to a zero-risk score and first check for malicious URLs via safe browsing API client. If the API returns a safe response, the system falls back to a heuristic keyword scan, incrementing the risk score before outputting the final visual veridict this is good as it ensure that logic ensure that there is high accuracy while preventing false negatives.

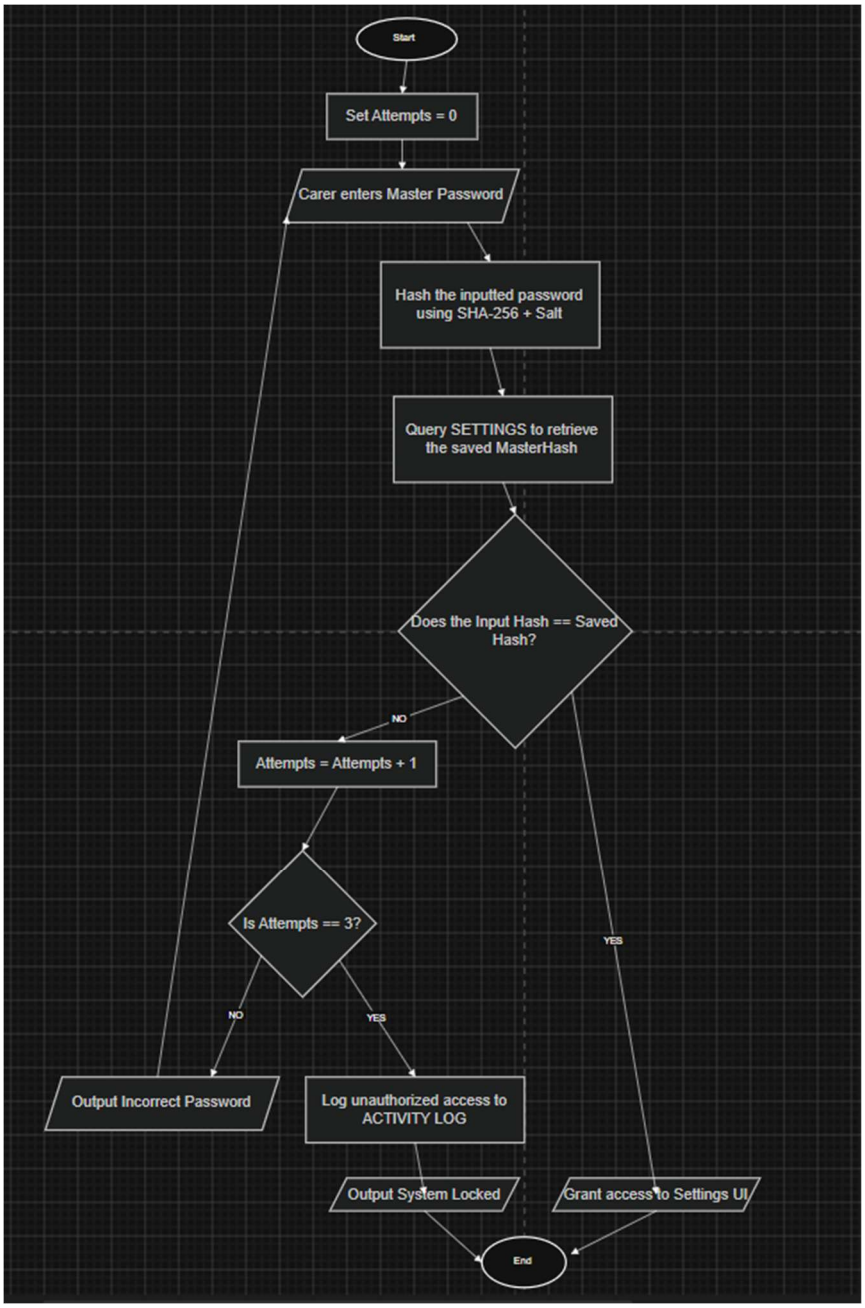
- Password vault:



This flowchart outlines the data validation and encryption process for saving user credentials. As you can se from th flowchart I have utilized a WHILE loop to enforce defensive programming , the system will trap the user in the validation stage until their password is meet the strict security criteria and will also ensure I explain this to them in simplified terms so they wont get confused . once validated, the

flowchart show the exact sequence of AES-256 encryption format I will integrate before the SQL insert command is executed

-
- Carer settings:



This flowchart maps a more secure administrative gateway. It visualize the security loop that hashes the user's input with a cryptographic salt, compares it to the SQLite database, and enforces a strict three attempt lockout mechanism. So there structuring the algorithm this way explicitly protects the

system against brute-force attacks, logging any unauthorized attempts to the activity log before locking the system down

Algorithms:

To fully describe the solution, I have used a hybrid approach of using both flowchart and pseudocode. The flowchart earlier provide a clear , visualization representation of the user journey and system branching, while the pseudocode will demonstrate the complex backend behind the computation logic, including API agent request, cryptographic hashing and SQLdata integration

- threat Scanner:

```
BEGIN threat scanner
        userInput = input("paste your message or link here")
        riskScore =  0
        ExtractedURL = extractURL(userInput)

        If extractedURL != "" then
            ApiResult = checkSafeBrowsing(extractedURL)
            If apiResult == "MALICIOUS" then
                saveTOactivityLog(extractedURL, "malicious link")
            print("Danger – do not click this link it has been reported as harmful")

    stop
            end if
        end if
scamwords = ["urgent", "verify", "suspened", "click here". "gift card", "bank details"]
for each word in scamWords
        if word is found in userInput then
            riskScore = riskScore + 1
        end if
```

- Password vault:

```
BEGIN save password
        websiteName  = input("enter website name")
        username = input("enter your username")
        password = input("enter your password")
        validPassowrd  = false
        while validPassword  == false
        while  validPassword == false
            if len(password) < 0 then
                print("your password is too short please make it at least 8 character
                    with a number)
                password  = input ("enter your password again")
            else if password has no numbers then
                print("your password needs at least one number to keep t safe ")
                password = input("enter your password again")
            else
                validPassword = True
                print("strong password")
            end if
        end while
        encryptedPaasoword = AES256encrypt(password)
        saveToDatabase(websiteName, username, encryptedPassword)
        print("saved susccessfully your password is stored safely")
```

END

- Carer settings Login

```
BEGIN carer login
        Attempts = 0
        While attempts <  3
            Username = input("enter username")
            Password = input ("enter master password")

            Salt  = getFromDatabase("passwordSalt")
            inputHash = hash(password + salt )
            storedHash = getFromDatabase("masterhash")

            if inputHash == storedHash then
                print("access granted welcome to career settings")
                loadCarersetting()
                stop
            else
                attempts = attempts + 1
                saveToactivityLog("wrong password attempt")
                print("incorrect password you have " ( 3 – attempts) + "attempts left")
            end if
        end while
        print("this  area is now locked too many attempts")
END
```

Testing:

Testing is a crucial part of developing my KomeCare software. I will carry throughout my development process and also after the system has been fully completed. Testing during development allows me to identify and fix bugs as I build each module one at a time, which directly link to my decomposition approach. Testing after development allows me to check the system works as whole.

I will also carry out four different types of testing:

- White box testing – carried out by me as the developer with full knowledge of the internal code. This check that the logic, loops, branches and validation inside the code all work correctly.
- Black box testing – carried out by carers and family members who have no knowledge of the internal code. This checks that the system behaves correctly from user perspective
- Functional testing – checks that each specific features of the system delievers the correct output and meets my success criteria
- End user testing – carried out by elderly users aged 65 and above after the full system is complete. This is the most important type of testing for this project because the whole purpose of the software is to work for this specific group of user

- White box testing

White box testing is carried out by me as a developer because I have full knowledge of how the internal code works. The purpose of this testing is to make sure all the logic, validation loops and database operations inside each module behave exactly as I planned.

Test type	Module	Description	Test data	Data type	Expected result	Links to Success criteria
W1	Launcher	Application launches and overrides the windows desktop in fullscreen kio	Run application on 4GB RAM test machine	Normal	Fullscreen confirmed, taskbar hidden alt-tab does not minimise app, loads within 5 seconds	SC1,SC3
W2	Launcher	All clickable buttons of the home screen meet the RNIB minimum size of 60x60 pixels	Check pixel dimension of all buttons in code	Normal	Every button confirmed at the minimum 60x60	SC3
W3	Threat scanner	A message containing 4 or more scam keywords correctly increments the risk score the Danger threshold and display	For example “ your account has been suspended, verify your bank detail urgently , click here now	Normal	Risk score RED Danger screen displayed threat logged to ACTIVITY LOG table with correct timestamp	SC2

		red on the screen				
W4	Threat scanner	A message containing only 2 scam keyword correctly and suspicious triggers the AMBER Warning screen rather than red	Example “This is an urgent reminder about your delivery”	Normal	Riskscore amber warning screen displayed threat logged to activity log	SC2
W5	Threat scanner	A completely clean message with no scam keywords returns GREEN safe verdict and risk score stays at 0	For example “HI haolat hope you are well see you Thursday for tea”	Normal	Riskscore GREEN SAFE screen displayed nothing logged to activity log	SC2
W6	Threat scanner	A URL extracted from a message is correctly identified as typosquatting when it	http://amaZON.co.uk/login compared against trusted domain “amazon.co.uk”	Normal	Similarity score calculated and if it is below trusted threshold. Riskscore	SC2

		closely resembles a trusted domainn			incremented AMBER or RED verdict shown depending on total score	
W7	Password vault	A password meeting all requirement is correctly encrypted before being written to the SQLite database.	Website: “NHS Portal” Username: margaret@example.com Password: “Garden12”	Normal	Record saved to VAULT table. Inspecting raw database confirm cipher text and not stored as “Garden12”	SC4
W8	Password vault	A password with no numbers trigger s he correct warning and loops repeats and full explanation why	For example – password: “gardenfower”	Erroneous	Warning shown: “your password needs at least a number to keep it safe “loops repeats until valid password entered.	SC4
W9	Carer login	Three consecutive wrong	Wrong password entered three	Erroneous	After the third attempts system	SC5

		password locks the system logs all three attempts to the activity log and displays the locked message			locked message displayed.3 record visible in ACTIVITY LOG table NO access granted	
W10	Diary tool	A valid reminder saves correctly and appears in the correct chronological position in the list based on the SQL Order By command	Task “Take afternoon medication”. Date 2026-06-01 time 14:00	Normal	Reminder saved to REMINDER table. List refreshes and displays reminder in correct chronological position	SC3
W11	Diary tool	A date entered in the wrong format is caught by the format check will be blocked	Date: 01-06-2026 instead of correct format	Erroneous	Message shown “please enter the date correctly” no record saved NO crash	SC3

		before it is saved				

Black Box Testing:

Black box testing is carried our by carers and family members who have no knowledge of how the system has been built internally. The purpose of this type of tesing is to check the system behaves correctly form a user perspective and to confirm that all messages and outputs are written in plain English that a non technical person will understand.

Test Type	Area being tested	Description	Test data	Expected result	Links to SC
B1	Home screen	Tester views the home screen and confirm all buttons are clearly labelled, text is large enough to read and no technical jargon is visible	So they will view the home screen without any guidance from me	Tester confirms buttons are large. Clearly labelled in plain English and the screen does not feel overwhelming	SC3,SC7
B2	Check message (DANGER)	Tester check message tool and scans it	Dear customer your HMRC account will be suspended in 24 hours click here to verify your details your detail immediately	RED DANGER screen displayed with a plain English explanation. Tester understand the result without needing any further technical knowledge	SC2,SC7
B3	Check message SAFE	Tester a clearly safe personal messages and scans it	“ Hi Dad just me checking up on you, are you okay I will call	GREEN SAFE screen displayed tester feels reassured and	SC2,SC7

			you Sunday. Love janet”	understands the result immediately	
B4	Check message WARNING	Test borderline message that contain one suspicious phrase	This is an urgent reminder about your parcel delivery	Amber warning screen displayed with plain English advice	SC2,SC7
B5	Password vault	Tester add a new password entry and then views it in the vault list	Website name “NHS login”, Username: susan@example.com Password: Susan276“	Entry appears in vault list as a clearly labelled car. Tester can identify which account it belongs to without confusion	SC4,SC7
B6	Diary tool	Diary tool tester adds a new reminder using the diary screen without any guidance	Evening medication “Date: 2026-06-10, Time:19:00	Reminder appears in the list in the correct position. Tester confirms the screen is straightforward to use	SC3
B7	Navigation	Tester navigates to every screen in the application and attempts to find the home button on each	Test clicks through all screens freely	Home button found in the same position on every single screen. This will enables that the tester never feel stuck or confused about how to go back	SC8
B8	Carer settings	Carer tester enter the correct master password to access the settings area	Correct master password provided to carer before test	Settings screen loads. Carer will able to successfully adjust font size an other stuff and it will reflect in the main screen	SC5

B9	No technical jargon check	A carer and a non-technical tester read through every visible message label, button and alert across all the screen in the system	Full walkthrough of all screens by management	Management confirms no technical jargon is present on any screen that is visible to the elderly user	SC7

Functional Testing

Functional testing checks that each specific features of my system works correctly and meet my success criteria. Unlike white box testing this does not focus on the internal code structure , and unlike black box this testing will not be focused on user experience. it will specifically check whether each function porduces the correct output and performs within the required standard that was stated in my success section

Test type	Description	Expected result	Link to the success criteria
F1	Run a control batch of 20 known phishing messages and URLS through the threat scanner and count how many are correctly flagged as AMBER or RED	If at least 95% correctly flagged. This meets SC2 which requires at least 95% accuracy on a controlled test batch	SC2
F2	Programmatically check the pixel dimension of every button across all screens in the application	100% of button confirmed at minimum 60x60 pixel. Which means make the button big enough so no button falls below the RNIB minimum	SC3

F3	Save test password to the vault then inspect the raw SQLite database file directly to confirm encryption was applied before storage	I can clearly se the cipher text visible in the database. Plain text password will not be seen anywhere in the raw database file	SC4
F4	Attempt to access carer settings with 3 incorrect password in a row	System lock after exactly 3 attempt and give some time to reattempts . and all this 3 attempt will be recorded in the activity log with correct timestamps	SC5
F5	Saves 5 reminder in random date order and check the order that they will appear in the displayed list	All 5 reminder in the strict chronological order with the earliest task appearing at the top of the list	SC3
F6	Carer adjust font size to different sizes in the setting, application is fully closed and then reopened	Font size remains at the desire size being chosen after relaunch. Settings correctly saved to the local SQlite database and loaded on startup	SC8

End User testing

End user testing will carried out when after all the full system has been completed. It will be carried out by a small group of elderly users. This is the most important stage of testing for this project.

After each session user will be asked to complete a short feedback survey with rating scale questions the average scores from this survey will be directly compared against SC6 which requires an average case of use and confidence score of al least out 8 out of 10 .

Test No	Task given to user	How I will record result	Link to Success criteria
1	A message has been received . use the message tool to scan the message	Observe the use to if they can find the check message tool, paste the text and understand the result without help from me	SC6,SC7,SC8
2	Go to your diary and add a new reminder for your medication	Observe where the user can complete task independently and not how long it took them and see if they needed any extra guidance	SC6,SC8
3	Anew password for NHS account save it to the password section	Observe how they navigate to the vault, enter the details and save successfully without help	SC6,SC8
4	You are finished using the app shut down while using the app	Observe whether user can locate the shutdown button without any confusion or further help	SC6,SC8
5	Complete this short feedback form rating how easy the app was to use and if they felt more confident	Collect forms then calculate average across all participant. Compare against the sc6 threshold of 8/10	SC6